

Section 2.2 - Game-Level Analysis: Stacked DiD and Synthetic Control DiD

Rocky Rowell

2026-06-30

Description

This is the fourth and final analysis file for the project. It builds the two difference-in-differences models used in the thesis. Both designs add a control group, which is what the baseline model in Section 2.1 lacks. The first is a stacked DiD, where each treated game is paired with one matched Steam-Only control. The second is a synthetic control DiD, where each treated game is paired with a weighted combination of Steam-Only donor games, followed by a pooled DiD across the treated-synthetic pairs. Both are estimated at 60, 90, and 180 days around each game's Epic entry date.

These models gave me the most trouble of anything in the project, so this file carries far more diagnostic code than the other analysis files. I opted to keep these code snippets included.

Notes / Important Information

- Like all other sections, outcomes are price, sale frequency, and discount depth. A fourth outcome, discount depth conditional on a sale being active, is estimated as an exploratory check and does not appear in the thesis.
 - Discount depth averages in zeros on non-sale days, so it is close to the product of sale frequency and conditional depth. The conditional measure separates discounting more often from discounting more deeply.
 - The 30-day window used in Section 2.1 is dropped here. At weekly aggregation it leaves only four pre-treatment weeks, which is too thin to fit a trajectory against.
 - Synthetic control weights are solved exactly with quadratic programming. An earlier version used hand-written gradient descent that was silently returning something close to the uniform donor average. The Extra section at the back documents how I found this and confirms the replacement works.
 - Matching is done with replacement, so a Steam-Only game can serve as the control for more than one treated game. Section 4.7 reports how often this happens.
 - Weights and matches are fitted separately for each outcome rather than being fitted on price and reused.
-

0. Setup

0.1 Libraries

```
library(tidyverse)
library(fixest)
library(broom)
```

```
library(ggplot2)
library(modelsummary)
library(quadprog)
library(MASS, include.only = "ginv")
library(flextable)
library(officer)
select <- dplyr::select # MASS masks select()
```

0.2 Load Data

```
data <- read_csv("Data/cleaned_data.csv")
```

0.3 Epic Entry Dates

```
epic_entry_dates <- data %>%
  filter(shop_name == "Epic") %>%
  group_by(game_title) %>%
  summarise(entry_date = min(date), .groups = "drop")
```

0.4 Globals

Each outcome is listed alongside the outcome its controls are matched on and whether it is a primary result. Conditional discount depth is undefined in weeks with no sale days, which makes it too sparse to match on directly, so it borrows the discount depth match.

```
genre_cols <- c("RPG", "Action", "Adventure", "Shooter", "Strategy", "Simulation",
               "Platformer", "Puzzle", "Horror", "OpenWorld", "Survival",
               "Roguelike", "Multiplayer", "Narrative")

# Every outcome, the outcome its controls are matched on, and whether it is primary.
# disc_cond is too sparse (undefined in weeks with no sale days) to match on
# directly, so it borrows the discount-depth match.
outcome_spec <- tibble::tribble(
  ~col,      ~label,              ~match_on, ~primary,
  "price",   "Price (USD)",       "price",   TRUE,
  "sale",    "Sale Frequency",    "sale",    TRUE,
  "disc",    "Discount Depth (%)", "disc",    TRUE,
  "disc_cond", "Conditional Discount Depth (%)", "disc",    FALSE
)
outcome_cols <- outcome_spec$col
outcome_labels <- outcome_spec$label
match_keys <- unique(outcome_spec$match_on)

# Analysis windows. 30d is dropped: at weekly aggregation it yields
# floor(30/7) = 4 pre-treatment weeks, and min_pre_weeks = max(4, .) requires 4,
# so every pre-week must be present. Too thin to fit trajectories.
windows <- c(60, 90, 180)

# Panel half-width. Set to 2 x the widest analysis window so the in-time placebo
# (fake entry at true_entry - w) has pre-fake-entry data to match on.
panel_halfwidth <- 2 * max(windows)
```

1. Panel Construction

1.1 Build the Steam Panel

```
steam_raw <- data %>%
  filter(shop_name == "Steam") %>%
  left_join(epic_entry_dates, by = "game_title") %>%
  mutate(
    days_since_entry = as.numeric(date - entry_date),
    ever_treated     = !is.na(entry_date),
    post             = if_else(ever_treated, days_since_entry >= 0, FALSE)
  ) %>%
  filter(!ever_treated | abs(days_since_entry) <= panel_halfwidth)

steam_panel <- steam_raw %>%
  group_by(game_title, date) %>%
  summarise(
    price      = mean(price,          na.rm = TRUE),
    sale       = mean(sale,           na.rm = TRUE),
    disc       = mean(discount_percent, na.rm = TRUE),
    # Conditional depth: mean discount on days the game IS on sale.
    # NaN when the game had no sale rows that day; converted to NA below.
    disc_cond  = mean(discount_percent[sale == 1], na.rm = TRUE),
    entry_date = first(entry_date),
    release_date = first(release_date),
    days_since_entry = first(days_since_entry),
    ever_treated = first(ever_treated),
    post       = first(post),
    across(all_of(genre_cols), first),
    .groups    = "drop"
  ) %>%
  mutate(across(all_of(outcome_cols), ~ if_else(is.nan(.x), NA_real_, .x)))
```

1.2 Trim Controls to the Treated Calendar Range

```
window_min <- steam_panel %>% filter(ever_treated) %>% summarise(m = min(date)) %>% pull(m)
window_max <- steam_panel %>% filter(ever_treated) %>% summarise(m = max(date)) %>% pull(m)

steam_panel <- steam_panel %>%
  filter(ever_treated | (date >= window_min & date <= window_max))
```

1.3 Data Corrections

Two hand corrections are applied before any analysis. Both are recorded here rather than in the cleaning file, since undocumented exclusions read as researcher degrees of freedom even when they are entirely legitimate.

```
# CORRECTION - Book of Demons.
# IsThereAnyDeal recorded a small number of sub-$5 observations for this title
# on dates when the Steam store page listed $4.99. These are recording errors
# rather than real price movements, so they are reset to the listed price.
steam_panel <- steam_panel %>%
  mutate(price = if_else(game_title == "Book of Demons" & price < 5, 4.99, price))

# EXCLUSIONS - three titles whose base-price series is not comparable across
```

```

# the entry window.
#   The Sims 4 - converted to free-to-play in October 2022, so the base price
#               outcome is undefined for much of the window.
#   Dying Light - TODO: record reason before publishing.
#   Grounded    - TODO: record reason before publishing.
dropped_titles <- c("Dying Light", "Grounded", "The Sims 4")
steam_panel <- steam_panel %>% filter(!game_title %in% dropped_titles)

```

1.4 Sample Flow

Tracks how many games survive each stage, from the raw pull down to the treated sample and the Steam-Only control pool. Retention into each individual DiD window is reported separately in Sections 4.5 and 5.5.

```

sample_flow <- tibble::tribble(
  ~stage,                                ~n_games,
  "All games in raw data",               n_distinct(data$game_title),
  "Ever listed on Epic (Dual-Platform)", n_distinct(epic_entry_dates$game_title),
  "After hand exclusions",               n_distinct(steam_panel$game_title[steam_panel$ever_treated]),
  "Steam-Only (control pool)",           n_distinct(steam_panel$game_title[!steam_panel$ever_treated])
)
sample_flow

```

```

## # A tibble: 4 x 2
##   stage                                n_games
##   <chr>                                <int>
## 1 All games in raw data                900
## 2 Ever listed on Epic (Dual-Platform)  356
## 3 After hand exclusions                326
## 4 Steam-Only (control pool)           542

```

2. Shared Matching Objects

Both estimators draw on the same donor pool and the same weekly series, so those objects are built once here. Genres are converted to a matrix so that donor eligibility can be checked with a single matrix multiply.

```

origin <- min(steam_panel$date)

game_attrs <- data %>%
  group_by(game_title) %>%
  summarise(release_date = first(release_date),
            across(all_of(genre_cols), first), .groups = "drop")

treated_info <- steam_panel %>%
  filter(ever_treated) %>%
  distinct(game_title, entry_date) %>%
  left_join(game_attrs, by = "game_title")

entry_lookup <- setNames(treated_info$entry_date, treated_info$game_title)

control_ranges <- steam_panel %>%
  filter(!ever_treated) %>%
  group_by(game_title) %>%

```

```

  summarise(min_date = min(date), max_date = max(date), .groups = "drop") %>%
  left_join(game_attrs, by = "game_title")

control_genre_mat <- control_ranges %>% select(all_of(genre_cols)) %>% as.matrix()

# Weekly calendar series for ALL outcomes (treated + control)
to_weekly <- function(df) {
  df %>%
    mutate(cal_week = as.integer(floor(as.numeric(date - origin) / 7))) %>%
    group_by(game_title, cal_week) %>%
    summarise(across(all_of(outcome_cols), ~ mean(.x, na.rm = TRUE)), .groups = "drop") %>%
    mutate(across(all_of(outcome_cols), ~ if_else(is.nan(.x), NA_real_, .x)))
}

treated_weekly_cal <- steam_panel %>% filter(ever_treated) %>% to_weekly()
control_weekly_cal <- steam_panel %>% filter(!ever_treated) %>% to_weekly()

```

3. Weight Solver

Solves the constrained least-squares problem for the synthetic control weights, subject to the weights being non-negative and summing to one. With demeaning turned on, the fit targets the treated game's trend rather than its level, since the level gap is absorbed by the sub-experiment fixed effects anyway.

This replaced a hand-written gradient descent routine that was not converging. The diagnostic that caught it, along with the old solver itself, is kept in the Extra section at the back of this file.

```

solve_synt_weights <- function(Y_pre, X_pre, demean = TRUE, ridge = 1e-6) {
  X_pre <- as.matrix(X_pre)
  J <- ncol(X_pre)
  if (J < 1) return(numeric(0))
  if (J == 1) return(1)

  if (demean) {
    Yc <- Y_pre - mean(Y_pre)
    Xc <- sweep(X_pre, 2, colMeans(X_pre), "-")
  } else {
    Yc <- Y_pre
    Xc <- X_pre
  }

  Dmat0 <- crossprod(Xc)
  dvec <- as.vector(crossprod(Xc, Yc))
  Amat <- cbind(rep(1, J), diag(J)) # col 1: sum(w) = 1 (equality); rest: w >= 0
  bvec <- c(1, rep(0, J))
  scale_d <- max(mean(diag(Dmat0)), 1e-8)

  # X'X is singular whenever J > T_pre, so escalate the ridge until PD.
  for (mult in c(1, 10, 100, 1e3, 1e4, 1e5, 1e6)) {
    Dmat <- Dmat0 + diag(ridge * mult * scale_d, J)
    sol <- tryCatch(quadprog::solve.QP(Dmat, dvec, Amat, bvec, meq = 1),
      error = function(e) NULL)
    if (!is.null(sol)) {

```

```

    w <- pmax(sol$solution, 0)
    s <- sum(w)
    if (s > 0) return(w / s)
  }
}
rep(1 / J, J) # last-resort fallback (should not trigger)
}

```

4. Stacked Difference-in-Differences

4.1 Matcher

Each treated game is paired with the single Steam-Only game that best matches it. Trajectory distance is computed on demeaned series so it captures trend divergence, which is what the game fixed effects do not absorb. The level gap, genre overlap, and release-date distance enter as separate, lower-weighted criteria.

```

# Level is a SEPARATE, lower-weighted criterion; trajectory distance is
# computed on demeaned series so it captures TREND divergence, which is what the
# subexp^game_title FE does NOT absorb.

```

```

match_w <- c(traj = 1.0, level = 0.3, genre = 0.3, release = 0.1)

```

```

safe_z <- function(x) {
  s <- sd(x, na.rm = TRUE)
  if (is.na(s) || s == 0) return(rep(0, length(x)))
  (x - mean(x, na.rm = TRUE)) / s
}

```

```

match_controls <- function(w, n_pre_weeks, outcome = "price",
                           entry_map = entry_lookup,
                           games = treated_info$game_title) {
  min_overlap <- 1
  min_pre_weeks <- max(4, floor(n_pre_weeks / 2))

  map_dfr(games, function(tg) {
    tr <- treated_info %>% filter(game_title == tg)
    E <- entry_map[[tg]]
    entry_cal <- as.integer(floor(as.numeric(E - origin) / 7))
    pre_weeks <- (entry_cal - n_pre_weeks):(entry_cal - 1)
    tr_genres <- tr %>% select(all_of(genre_cols)) %>% unlist()

    t_path <- treated_weekly_cal %>%
      filter(game_title == tg, cal_week %in% pre_weeks) %>%
      transmute(cal_week, t_val = .data[[outcome]]) %>%
      filter(!is.na(t_val))
    if (nrow(t_path) < min_pre_weeks) return(NULL)

    shared <- as.integer(control_genre_mat %*% tr_genres)
    elig <- control_ranges %>%
      mutate(shared_genres = shared) %>%
      filter(shared_genres >= min_overlap,
             min_date <= E - w, max_date >= E + w)
    if (nrow(elig) == 0) return(NULL)
  })
}

```

```

cand <- control_weekly_cal %>%
  filter(game_title %in% elig$game_title, cal_week %in% pre_weeks) %>%
  transmute(game_title, cal_week, c_val = .data[[outcome]]) %>%
  filter(!is.na(c_val)) %>%
  inner_join(t_path, by = "cal_week") %>%
  group_by(game_title) %>%
  summarise(
    n_overlap = n(),
    # TREND distance (both series demeaned over the overlapping weeks)
    traj_mse = mean(((c_val - mean(c_val)) - (t_val - mean(t_val)))^2),
    # LEVEL gap, kept separate and down-weighted
    level_gap = abs(mean(c_val) - mean(t_val)),
    .groups = "drop"
  ) %>%
  filter(n_overlap >= min_pre_weeks) %>%
  left_join(elig %>% select(game_title, shared_genres, release_date),
    by = "game_title")
if (nrow(cand) == 0) return(NULL)

cand <- cand %>%
  mutate(
    z_traj = safe_z(traj_mse),
    z_level = safe_z(level_gap),
    z_genre = -safe_z(shared_genres),
    z_rel = safe_z(abs(as.numeric(release_date) - as.numeric(tr$release_date))),
    score = match_w["traj"] * z_traj +
      match_w["level"] * z_level +
      match_w["genre"] * z_genre +
      match_w["release"] * z_rel
  )

best <- cand %>% slice_min(score, n = 1, with_ties = FALSE)
tibble(
  treated_game = tg,
  control_game = best$game_title,
  traj_rmse = sqrt(best$traj_mse), # TREND rmse - comparable to synth pre_rmse_trend
  level_gap = best$level_gap,
  shared_genres = best$shared_genres,
  n_pre_overlap = best$n_overlap,
  n_eligible = nrow(cand)
)
})
}

```

4.2 Stacked Panel Builder

```

build_stacked <- function(matched, w, entry_map = entry_lookup) {
  n_weeks <- floor(w / 7)
  if (is.null(matched) || nrow(matched) == 0) return(NULL)

  map_dfr(seq_len(nrow(matched)), function(i) {
    tg <- matched$treated_game[i]

```

```

cg    <- matched$control_game[i]
entry <- entry_map[[tg]]

t_rows <- steam_panel %>%
  filter(game_title == tg,
         date >= entry - w, date <= entry + w) %>%
  transmute(subexp = tg, game_title, treated = 1L,
            across(all_of(outcome_cols)),
            days_since_entry = as.numeric(date - entry))

c_rows <- steam_panel %>%
  filter(game_title == cg, date >= entry - w, date <= entry + w) %>%
  transmute(subexp = tg, game_title, treated = 0L,
            across(all_of(outcome_cols)),
            days_since_entry = as.numeric(date - entry))

bind_rows(t_rows, c_rows)
}) %>%
  mutate(event_week = days_since_entry %/% 7) %>%
  filter(event_week >= -n_weeks, event_week <= n_weeks) %>%
  group_by(subexp, game_title, treated, event_week) %>%
  summarise(across(all_of(outcome_cols), ~ mean(.x, na.rm = TRUE)), .groups = "drop") %>%
  mutate(across(all_of(outcome_cols), ~ if_else(is.nan(.x), NA_real_, .x)),
         post = as.integer(event_week >= 0))
}

```

4.3 Model Runner and Joint Pre-Trend Test

The pre-trend test is a Wald chi-square on the joint nullity of every pre-period lead. A high p-value converts “the event study plot looks flat” into “parallel pre-trends are not rejected”.

```

# Wald chi-square on the joint nullity of ALL pre-period leads.
pretrend_test <- function(es) {
  nm <- names(coef(es))
  pre <- nm[grepl("event_week::-", nm)]
  if (!length(pre)) return(tibble(chi2 = NA_real_, df = NA_integer_, p_value = NA_real_))
  b <- coef(es)[pre]
  V <- vcov(es)[pre, pre, drop = FALSE]
  Vi <- tryCatch(solve(V), error = function(e) MASS::ginv(V))
  chi2 <- as.numeric(t(b) %*% Vi %*% b)
  tibble(chi2 = chi2, df = length(pre),
         p_value = pchisq(chi2, df = length(pre), lower.tail = FALSE))
}

run_one_stacked <- function(dat, oc) {
  fe <- "subexp~game_title + subexp~event_week"
  att <- feols(as.formula(paste0(oc, " ~ treated:post | ", fe)),
              data = dat, cluster = ~ game_title)
  es <- feols(as.formula(paste0(oc, " ~ i(event_week, treated, ref = -1) | ", fe)),
              data = dat, cluster = ~ game_title)
  list(att = att, es = es, pretrend = pretrend_test(es))
}

```

4.4 Run

```
did_results <- map(windows, function(w) {
  n_pre <- floor(w / 7)

  matched <- map(setNames(match_keys, match_keys), ~ match_controls(w, n_pre, .x))
  stacked <- map(matched, ~ build_stacked(.x, w))

  models <- map(setNames(outcome_cols, outcome_cols), function(oc) {
    key <- outcome_spec$match_on[outcome_spec$col == oc]
    run_one_stacked(stacked[[key]], oc)
  })

  list(window = w, matched = matched, stacked = stacked, models = models)
})
names(did_results) <- paste0("w", windows)
```

4.5 Match Quality

Trajectory RMSE here is measured on demeaned series, which makes it directly comparable to the synthetic control's pre-period trend RMSE in Section 5.5.

```
stacked_quality <- map_dfr(did_results, function(r) {
  map_dfr(match_keys, function(k) {
    m <- r$matched[[k]]
    tibble(
      window      = r$window,
      matched_on  = k,
      n_matched   = nrow(m),
      mean_traj_rmse = round(mean(m$traj_rmse, na.rm = TRUE), 3), # TREND rmse
      median_traj_rmse = round(median(m$traj_rmse, na.rm = TRUE), 3),
      mean_level_gap = round(mean(m$level_gap, na.rm = TRUE), 3),
      mean_shared   = round(mean(m$shared_genres), 2),
      mean_n_eligible = round(mean(m$n_eligible), 1)
    )
  })
})
stacked_quality
```

```
## # A tibble: 9 x 8
##   window matched_on n_matched mean_traj_rmse median_traj_rmse mean_level_gap
##   <dbl> <chr>      <int>      <dbl>      <dbl>      <dbl>
## 1    60 price        255      2.46      2.07      4.75
## 2    60 sale        255      0.065      0       0.034
## 3    60 disc        255      5.74      3.69      2.56
## 4    90 price        244      3.01      2.39      4.46
## 5    90 sale        244      0.091     0.058     0.036
## 6    90 disc        244      7.58      6.23      2.89
## 7   180 price        227      3.79      3.16      4.93
## 8   180 sale        227      0.152     0.16      0.044
## 9   180 disc        227     10.6     10.1      3.01
## # i 2 more variables: mean_shared <dbl>, mean_n_eligible <dbl>
```

4.6 Covariate Balance, Treated vs. Matched Control

```
balance_table <- map_dfr(did_results, function(r) {
  map_dfr(match_keys, function(k) {
    m <- r$matched[[k]]
    if (is.null(m) || nrow(m) == 0) return(NULL)

    # Iterate ROW-WISE so a control reused across sub-experiments is evaluated on
    # the correct treated game's pre-window each time.
    pair_means <- map_dfr(seq_len(nrow(m)), function(i) {
      E <- entry_lookup[[m$treated_game[i]]]
      tw <- steam_panel %>%
        filter(game_title == m$treated_game[i], date >= E - r$window, date < E) %>%
        summarise(across(all_of(outcome_cols), ~ mean(.x, na.rm = TRUE))) %>%
        mutate(grp = "Treated")
      cw <- steam_panel %>%
        filter(game_title == m$control_game[i], date >= E - r$window, date < E) %>%
        summarise(across(all_of(outcome_cols), ~ mean(.x, na.rm = TRUE))) %>%
        mutate(grp = "Matched control")
      bind_rows(tw, cw)
    })

    pair_means %>%
      group_by(grp) %>%
      summarise(across(all_of(outcome_cols), ~ round(mean(.x, na.rm = TRUE), 3)), .groups = "drop") %>%
      mutate(window = r$window, matched_on = k, .before = 1)
  })
})
balance_table
```

```
## # A tibble: 18 x 7
##   window matched_on grp           price  sale  disc disc_cond
##   <dbl> <chr>      <chr>         <dbl> <dbl> <dbl>      <dbl>
## 1     60 price    Matched control  24.8 0.157  8.56      54.4
## 2     60 price    Treated          26.8 0.203 10.9      57.4
## 3     60 sale    Matched control  23.0 0.182 10.6      57.3
## 4     60 sale    Treated          26.8 0.203 10.9      57.4
## 5     60 disc    Matched control  22.9 0.172  9.58      55.2
## 6     60 disc    Treated          26.8 0.203 10.9      57.4
## 7     90 price    Matched control  24.7 0.149  8.07      53.3
## 8     90 price    Treated          26.3 0.196 10.9      57.8
## 9     90 sale    Matched control  23.1 0.169  9.79      56.4
## 10    90 sale    Treated          26.3 0.196 10.9      57.8
## 11    90 disc    Matched control  22.7 0.166  9.29      53.9
## 12    90 disc    Treated          26.3 0.196 10.9      57.8
## 13   180 price    Matched control  23.4 0.154  8.17      52.8
## 14   180 price    Treated          25.9 0.191 10.8      58.2
## 15   180 sale    Matched control  22.3 0.159  9.23      55.5
## 16   180 sale    Treated          25.9 0.191 10.8      58.2
## 17   180 disc    Matched control  23.2 0.16   8.72      53.5
## 18   180 disc    Treated          25.9 0.191 10.8      58.2
```

4.7 Control Reuse

Matching is done with replacement, so this quantifies how concentrated the control pool ends up being.

```
reuse_table <- map_dfr(did_results, function(r) {  
  map_dfr(match_keys, function(k) {  
    m <- r$matched[[k]]  
    tab <- table(m$control_game)  
    tibble(  
      window          = r$window,  
      matched_on      = k,  
      n_treated       = nrow(m),  
      n_unique_controls = length(tab),  
      max_reuse       = max(tab),  
      mean_reuse      = round(mean(tab), 2),  
      pct_controls_used_once = round(100 * mean(tab == 1), 1)  
    )  
  })  
})  
reuse_table
```

```
## # A tibble: 9 x 7  
##   window matched_on n_treated n_unique_controls max_reuse mean_reuse  
##   <dbl> <chr>      <int>          <int>          <int>      <dbl>  
## 1     60 price         255            165           5        1.55  
## 2     60 sale         255            168           5        1.52  
## 3     60 disc         255            157           6        1.62  
## 4     90 price         244            148           7        1.65  
## 5     90 sale         244            156           6        1.56  
## 6     90 disc         244            164           5        1.49  
## 7    180 price         227            145           6        1.57  
## 8    180 sale         227            155           6        1.46  
## 9    180 disc         227            162           5        1.4  
## # i 1 more variable: pct_controls_used_once <dbl>
```

5. Synthetic Control DiD

Each treated game is matched to a weighted combination of Steam-Only donors fitted on its pre-entry trend, and the treated-synthetic pairs are then pooled into a single DiD. This follows the constrained regression approach in Doudchenko and Imbens (2016), with the simplex-constrained weights of Abadie, Diamond and Hainmueller (2010) and pooling across treated units as in Cavallo, Galiani, Noy and Pantano (2013). It is not the synthetic difference-in-differences of Arkhangelsky et al. (2021), which also estimates time weights.

5.1 Synthetic Control Builder

```
make_synth <- function(tg, w, n_weeks, fit_on = "price", apply_to = fit_on,  
  entry_map = entry_lookup) {  
  tr      <- treated_info %>% filter(game_title == tg)  
  E      <- entry_map[[tg]]  
  entry_cal <- as.integer(floor(as.numeric(E - origin) / 7))  
  tr_genres <- tr %>% select(all_of(genre_cols)) %>% unlist()  
  
  pre_weeks <- (entry_cal - n_weeks):(entry_cal - 1)
```

```

full_weeks <- (entry_cal - n_weeks):(entry_cal + n_weeks)
min_pre <- max(4, floor(n_weeks / 2))

t_full <- treated_weekly_cal %>%
  filter(game_title == tg, cal_week %in% full_weeks) %>%
  arrange(cal_week)
if (nrow(t_full) == 0) return(NULL)

y_fit_all <- t_full[[fit_on]]
if (sum(t_full$cal_week %in% pre_weeks & !is.na(y_fit_all)) < min_pre) return(NULL)

shared <- as.integer(control_genre_mat %*% tr_genres)
elig <- control_ranges %>%
  mutate(shared_genres = shared) %>%
  filter(shared_genres >= 1, min_date <= E - w, max_date >= E + w) %>%
  pull(game_title)
n_elig <- length(elig)
if (n_elig < 3) return(NULL)

donor_data <- control_weekly_cal %>%
  filter(game_title %in% elig, cal_week %in% full_weeks)

# ---- donor matrix used to FIT the weights (must be complete) ----
fit_wide <- donor_data %>%
  transmute(game_title, cal_week, value = .data[[fit_on]]) %>%
  pivot_wider(names_from = game_title, values_from = value) %>%
  arrange(cal_week)

Xall <- fit_wide %>% select(-cal_week)
keep <- colSums(is.na(Xall)) == 0
n_kept <- sum(keep)
if (n_kept < 3) return(NULL)
donors <- names(Xall)[keep]
Xall <- as.matrix(Xall[, keep, drop = FALSE])

weeks <- fit_wide$cal_week
t_idx <- match(weeks, t_full$cal_week)
Y_fit <- y_fit_all[t_idx]
pre_idx <- which(weeks %in% pre_weeks)
ok <- pre_idx[!is.na(Y_fit[pre_idx])]
if (length(ok) < min_pre) return(NULL)

Yp <- Y_fit[ok]
Xp <- Xall[ok, , drop = FALSE]

# Exact QP on DEMEANED series
wts <- solve_synt_h_weights(Yp, Xp, demean = TRUE)

fit_pre <- as.vector(Xp %*% wts)
pre_rmse_trend <- sqrt(mean(((fit_pre - mean(fit_pre)) - (Yp - mean(Yp)))^2))
pre_rmse_level <- sqrt(mean((fit_pre + (mean(Yp) - mean(fit_pre)) - Yp)^2))

# ---- apply the SAME weights to the target outcome ----

```

```

app_wide <- donor_data %>%
  transmute(game_title, cal_week, value = .data[[apply_to]]) %>%
  pivot_wider(names_from = game_title, values_from = value) %>%
  arrange(cal_week)
app_wide <- app_wide[match(weeks, app_wide$cal_week), , drop = FALSE]
Aapp <- as.matrix(app_wide[, donors, drop = FALSE])

# NA-safe weighted mean (renormalise over donors available that week)
synth_raw <- apply(Aapp, 1, function(rw) {
  o <- !is.na(rw)
  if (!any(o)) return(NA_real_)
  sum(rw[o] * wts[o]) / sum(wts[o])
})

Y_app <- t_full[[apply_to]][t_idx]
ok_app <- pre_idx[!is.na(Y_app[pre_idx]) & !is.na(synth_raw[pre_idx])]
if (length(ok_app) < 2) return(NULL)

# Restore the level using the APPLIED outcome's pre-period
offset <- mean(Y_app[ok_app]) - mean(synth_raw[ok_app])

tibble(
  subexp      = tg,
  cal_week    = weeks,
  event_week  = weeks - entry_cal,
  t_val       = Y_app,
  s_val       = synth_raw + offset,
  pre_rmse_trend = pre_rmse_trend,
  pre_rmse_level = pre_rmse_level,
  n_elig      = n_elig,
  n_donors    = n_kept,      # survivors of the complete-case filter
  max_weight  = max(wts),
  eff_donors  = sum(wts > 0.01)
) %>% filter(!is.na(t_val), !is.na(s_val))
}

```

5.2 Panel Assembly

```

build_synth_panel <- function(w, fit_on, apply_to = fit_on, entry_map = entry_lookup,
                             games = treated_info$game_title) {
  n_weeks <- floor(w / 7)
  lst <- map(games, ~ make_synth(.x, w, n_weeks, fit_on, apply_to, entry_map))
  lst <- compact(lst)
  if (!length(lst)) return(NULL)

  panel <- map_dfr(lst, function(s) {
    bind_rows(
      s %>% transmute(subexp, event_week, treated = 1L, y = t_val),
      s %>% transmute(subexp, event_week, treated = 0L, y = s_val)
    )
  }) %>% mutate(post = as.integer(event_week >= 0))

  diag <- map_dfr(lst, ~ distinct(.x, subexp, pre_rmse_trend, pre_rmse_level,

```

```

                                n_elig, n_donors, max_weight, eff_donors))
  list(panel = panel, diag = diag)
}

```

5.3 Model Runner

```

run_one_synth <- function(panel) {
  fe <- "subexp~treated + subexp~event_week"
  att <- feols(as.formula(paste0("y ~ treated:post | ", fe)),
    data = panel, cluster = ~ subexp)
  es <- feols(as.formula(paste0("y ~ i(event_week, treated, ref = -1) | ", fe)),
    data = panel, cluster = ~ subexp)
  list(att = att, es = es, pretrend = pretrend_test(es))
}

```

5.4 Run

```

synth_results <- map(windows, function(w) {
  by_outcome <- map(setNames(outcome_cols, outcome_cols), function(oc) {
    fo <- outcome_spec$match_on[outcome_spec$col == oc]
    sp <- build_synth_panel(w, fit_on = fo, apply_to = oc)
    if (is.null(sp)) return(NULL)
    list(models = run_one_synth(sp$panel), diag = sp$diag, panel = sp$panel)
  })
  list(window = w, by_outcome = by_outcome)
})
names(synth_results) <- paste0("w", windows)

```

5.5 Synthetic Control Diagnostics

```

synth_quality <- map_dfr(synth_results, function(r) {
  map_dfr(outcome_cols, function(oc) {
    x <- r$by_outcome[[oc]]
    if (is.null(x)) return(NULL)
    d <- x$diag
    tibble(
      window          = r$window,
      outcome          = oc,
      fitted_on        = outcome_spec$match_on[outcome_spec$col == oc],
      n_synth          = nrow(d),
      mean_pre_rmse_trend = round(mean(d$pre_rmse_trend, na.rm = TRUE), 3), # <- the slide number
      med_pre_rmse_trend = round(median(d$pre_rmse_trend, na.rm = TRUE), 3),
      mean_pre_rmse_level = round(mean(d$pre_rmse_level, na.rm = TRUE), 3),
      mean_n_eligible    = round(mean(d$n_elig), 1),
      mean_n_donors_kept = round(mean(d$n_donors), 1),
      pct_donors_lost    = round(100 * (1 - mean(d$n_donors / d$n_elig)), 1),
      mean_max_weight    = round(mean(d$max_weight), 3),
      mean_eff_donors    = round(mean(d$eff_donors), 1)
    )
  })
})
synth_quality

```

```
## # A tibble: 12 x 12
##   window outcome   fitted_on n_synth mean_pre_rmse_trend med_pre_rmse_trend
##   <dbl> <chr>     <chr>      <int>      <dbl>          <dbl>
## 1    60 price     price       255        0.737           0
## 2    60 sale     sale       255        0.019           0
## 3    60 disc     disc       255        1.73            0
## 4    60 disc_cond disc       161        2.62            0
## 5    90 price     price       244        1.09            0
## 6    90 sale     sale       244        0.038           0
## 7    90 disc     disc       244        2.87            0
## 8    90 disc_cond disc       188        3.62            0.888
## 9   180 price     price       227        2.01            1.16
## 10  180 sale     sale       227        0.089           0.082
## 11  180 disc     disc       227        5.67            4.16
## 12  180 disc_cond disc       205        6.22            4.93
## # i 6 more variables: mean_pre_rmse_level <dbl>, mean_n_eligible <dbl>,
## #   mean_n_donors_kept <dbl>, pct_donors_lost <dbl>, mean_max_weight <dbl>,
## #   mean_eff_donors <dbl>
```

Pre-Trend Fit: Stacked vs. Synthetic

The ratio compares how well each estimator tracks the treated game's pre-treatment trend. A value above one means the synthetic control fits better than the single matched control, which is the condition under which the synthetic estimates are preferred. A value below one would favor the stacked estimates.

```
fit_comparison <- stacked_quality %>%
  select(window, matched_on, stacked_trend_rmse = mean_traj_rmse, n_matched) %>%
  left_join(
    synth_quality %>%
      filter(outcome == fitted_on) %>%
      select(window, matched_on = fitted_on,
             synth_trend_rmse = mean_pre_rmse_trend, n_synth),
    by = c("window", "matched_on")
  ) %>%
  mutate(synth_better_by = round(stacked_trend_rmse / synth_trend_rmse, 2))
fit_comparison
```

```
## # A tibble: 9 x 7
##   window matched_on stacked_trend_rmse n_matched synth_trend_rmse n_synth
##   <dbl> <chr>          <dbl>      <int>      <dbl>      <int>
## 1    60 price         2.46       255        0.737       255
## 2    60 sale         0.065      255        0.019       255
## 3    60 disc         5.74       255        1.73        255
## 4    90 price         3.01       244        1.09        244
## 5    90 sale         0.091      244        0.038       244
## 6    90 disc         7.58       244        2.87        244
## 7   180 price         3.79       227        2.01        227
## 8   180 sale         0.152      227        0.089       227
## 9   180 disc        10.6        227        5.67        227
## # i 1 more variable: synth_better_by <dbl>
```

6. Results

6.1 Pooled ATT

```
comparison <- bind_rows(
  map_dfr(did_results, function(r) {
    map_dfr(outcome_cols, function(oc) {
      tidy(r$models[[oc]]$att) %>%
        filter(term == "treated:post") %>%
        transmute(method = "Stacked DiD", outcome = oc, window = r$window,
                  estimate, std.error, statistic, p.value,
                  n_obs = nobs(r$models[[oc]]$att))
    })
  }),
  map_dfr(synth_results, function(r) {
    map_dfr(outcome_cols, function(oc) {
      x <- r$by_outcome[[oc]]
      if (is.null(x)) return(NULL)
      tidy(x$models$att) %>%
        filter(term == "treated:post") %>%
        transmute(method = "Synthetic Control DiD", outcome = oc, window = r$window,
                  estimate, std.error, statistic, p.value,
                  n_obs = nobs(x$models$att))
    })
  })
) %>%
left_join(outcome_spec %>% select(outcome = col, label), by = "outcome") %>%
mutate(
  ci_lo = estimate - 1.96 * std.error,
  ci_hi = estimate + 1.96 * std.error,
  sig = case_when(p.value < .001 ~ "***", p.value < .01 ~ "**",
                 p.value < .05 ~ "*", TRUE ~ "")
) %>%
arrange(outcome, window, method)

comparison %>%
mutate(across(c(estimate, std.error, ci_lo, ci_hi), ~ round(.x, 3))) %>%
select(label, window, method, estimate, std.error, ci_lo, ci_hi, sig, p.value, n_obs) %>%
print(n = 100)
```

```
## # A tibble: 24 x 10
##   label      window method estimate std.error ci_lo ci_hi sig  p.value n_obs
##   <chr>      <dbl> <chr>    <dbl>    <dbl> <dbl> <dbl> <chr>    <dbl> <int>
## 1 Discount D~    60 Stack~  -0.119    0.605  -1.30  1.07  ""     0.844   8596
## 2 Discount D~    60 Synth~   0.5      0.56   -0.597  1.60  ""     0.373   8586
## 3 Discount D~    90 Stack~  -0.048    0.549  -1.12  1.03  ""     0.931  12110
## 4 Discount D~    90 Synth~   0.349    0.476  -0.584  1.28  ""     0.464  12104
## 5 Discount D~   180 Stack~   0.356    0.472  -0.57  1.28  ""     0.452  22986
## 6 Discount D~   180 Synth~   1.19     0.374   0.452  1.92  "***"  0.00174 22974
## 7 Conditiona~    60 Stack~  -0.908    0.786  -2.45  0.633  ""     0.249   1684
## 8 Conditiona~    60 Synth~   1.79     0.812   0.203  3.39  "*"    0.0285   2036
## 9 Conditiona~    90 Stack~   0.572    0.693  -0.786  1.93  ""     0.410   2390
## 10 Conditiona~   90 Synth~   1.29     0.783  -0.246  2.82  ""     0.102   3190
## 11 Conditiona~  180 Stack~   0.837    0.61   -0.358  2.03  ""     0.171   4398
```

```
## 12 Conditiona~ 180 Synth~ 2.25 0.689 0.902 3.60 "***" 0.00126 6666
## 13 Price (USD) 60 Stack~ 0.499 0.277 -0.044 1.04 "" 0.0724 8596
## 14 Price (USD) 60 Synth~ 0.348 0.248 -0.137 0.834 "" 0.161 8586
## 15 Price (USD) 90 Stack~ 0.475 0.231 0.023 0.927 "*" 0.0403 12110
## 16 Price (USD) 90 Synth~ 0.53 0.207 0.125 0.936 "*" 0.0110 12104
## 17 Price (USD) 180 Stack~ 0.138 0.199 -0.253 0.529 "" 0.490 22986
## 18 Price (USD) 180 Synth~ 0.233 0.196 -0.151 0.618 "" 0.236 22974
## 19 Sale Frequ~ 60 Stack~ -0.006 0.011 -0.027 0.014 "" 0.546 8596
## 20 Sale Frequ~ 60 Synth~ 0.009 0.012 -0.014 0.032 "" 0.448 8586
## 21 Sale Frequ~ 90 Stack~ -0.002 0.009 -0.019 0.016 "" 0.840 12110
## 22 Sale Frequ~ 90 Synth~ 0.009 0.009 -0.009 0.027 "" 0.330 12104
## 23 Sale Frequ~ 180 Stack~ 0.005 0.008 -0.01 0.02 "" 0.551 22986
## 24 Sale Frequ~ 180 Synth~ 0.018 0.01 -0.002 0.037 "" 0.0763 22974
```

The count below compares how many primary-outcome tests come back significant against how many would be expected by chance at the 5% level, which is the multiple-testing question quantified rather than argued.

```
# How many of the primary-outcome tests are significant, and how many would we
# expect by chance? (The multiple-testing defence, quantified.)
prim <- comparison %>% filter(outcome %in% outcome_spec$col[outcome_spec$primary])
tibble(
  n_tests      = nrow(prim),
  n_sig_05     = sum(prim$p.value < .05),
  expected_by_chance = round(0.05 * nrow(prim), 2),
  which_sig    = paste(with(prim[prim$p.value < .05, ],
                           paste0(label, "/", method, "/", window, "d")), collapse = "; ")
)
```

```
## # A tibble: 1 x 4
##   n_tests n_sig_05 expected_by_chance which_sig
##   <int>   <int>          <dbl> <chr>
## 1     18       3          0.9 Discount Depth (%) / Synthetic Control DiD/~
```

6.2 Joint Pre-Trend Tests

These p-values are reported alongside every event study. A low p-value indicates the counterfactual is a poor fit for that outcome and window.

```
pretrends <- bind_rows(
  map_dfr(did_results, function(r) {
    map_dfr(outcome_cols, function(oc) {
      r$models[[oc]]$pretrend %>%
        mutate(method = "Stacked DiD", outcome = oc, window = r$window)
    })
  }),
  map_dfr(synth_results, function(r) {
    map_dfr(outcome_cols, function(oc) {
      x <- r$by_outcome[[oc]]
      if (is.null(x)) return(NULL)
      x$models$pretrend %>%
        mutate(method = "Synthetic Control DiD", outcome = oc, window = r$window)
    })
  })
) %>%
left_join(outcome_spec %>% select(outcome = col, label), by = "outcome") %>%
mutate(chi2 = round(chi2, 2), p_value = round(p_value, 4),
```

```

      flag = if_else(p_value < 0.05, "PRE-TREND", "ok")) %>%
select(label, window, method, chi2, df, p_value, flag) %>%
arrange(label, window, method)

pretrends %>% print(n = 100)

## # A tibble: 24 x 7
##   label                window method      chi2    df p_value flag
##   <chr>                <dbl> <chr>    <dbl> <int>   <dbl> <chr>
## 1 Conditional Discount Depth (%)    60 Stacked DiD    6.85     7  0.444  ok
## 2 Conditional Discount Depth (%)    60 Synthetic Co~    4.39     7  0.734  ok
## 3 Conditional Discount Depth (%)    90 Stacked DiD    6.84    11  0.812  ok
## 4 Conditional Discount Depth (%)    90 Synthetic Co~   10.0    11  0.526  ok
## 5 Conditional Discount Depth (%)   180 Stacked DiD   26.6    24  0.324  ok
## 6 Conditional Discount Depth (%)   180 Synthetic Co~  25.3    24  0.390  ok
## 7 Discount Depth (%)              60 Stacked DiD    6.04     7  0.535  ok
## 8 Discount Depth (%)              60 Synthetic Co~    7.61     7  0.368  ok
## 9 Discount Depth (%)              90 Stacked DiD   11.3    11  0.415  ok
## 10 Discount Depth (%)             90 Synthetic Co~    8.63    11  0.656  ok
## 11 Discount Depth (%)            180 Stacked DiD   25.9    24  0.357  ok
## 12 Discount Depth (%)            180 Synthetic Co~  30.6    24  0.165  ok
## 13 Price (USD)                   60 Stacked DiD    9.61     7  0.212  ok
## 14 Price (USD)                   60 Synthetic Co~    7.27     7  0.402  ok
## 15 Price (USD)                   90 Stacked DiD   13.2    11  0.278  ok
## 16 Price (USD)                   90 Synthetic Co~   12.5    11  0.327  ok
## 17 Price (USD)                  180 Stacked DiD   30.7    24  0.161  ok
## 18 Price (USD)                  180 Synthetic Co~  43.7    24  0.0083 PRE--
## 19 Sale Frequency                60 Stacked DiD    3.72     7  0.811  ok
## 20 Sale Frequency                60 Synthetic Co~   12.0     7  0.101  ok
## 21 Sale Frequency                90 Stacked DiD    9.57    11  0.570  ok
## 22 Sale Frequency                90 Synthetic Co~   12.3    11  0.343  ok
## 23 Sale Frequency               180 Stacked DiD   22.2    24  0.567  ok
## 24 Sale Frequency               180 Synthetic Co~   35.8    24  0.0567 ok

```

The same results formatted as a display table for the thesis.

```

pretrend_wide <- pretrends %>%
  mutate(
    cell = sprintf("%.3f %s", p_value, if_else(p_value < 0.05, "(fail)", "(pass)"))
  ) %>%
  select(label, window, method, cell) %>%
  pivot_wider(names_from = method, values_from = cell) %>%
  filter(label != "Conditional Discount Depth (%)") %>% # drop the exploratory outcome
  mutate(label = factor(label, levels = c("Price (USD)", "Sale Frequency",
                                           "Discount Depth (%)"))) %>%

  arrange(label, window) %>%
  rename(Outcome = label, Window = window)

n_fail <- sum(pretrends$p_value < 0.05 &
  pretrends$label != "Conditional Discount Depth (%)", na.rm = TRUE)
n_test <- sum(pretrends$label != "Conditional Discount Depth (%)")

ft_pre <- flextable(pretrend_wide) %>%
  set_header_labels(Window = "Window (days)") %>%
  add_header_row(values = c("", "", "Joint pre-trend test (p-value)"),

```

```

colwidths = c(1, 1, 2), top = TRUE) %>%
add_footer_lines(sprintf(
  "H0: all pre-period event-study coefficients = 0 jointly (Wald chi-square, reference week = -1). Hi
  n_test - n_fail, n_test, 0.05 * n_test)) %>%
theme_booktabs() %>%
align(j = 2:4, align = "center", part = "all") %>%
align(align = "center", part = "header") %>%
bold(i = ~ Outcome == "Price (USD)" & Window == 90) %>%           # the main finding
merge_v(j = "Outcome") %>%
valign(j = "Outcome", valign = "top") %>%
fontsize(size = 9, part = "footer") %>%
autofit()

```

ft_pre

Outcome	Window (days)	Joint pre-trend test (p-value)	
		Stacked DiD	Synthetic Control DiD
Price (USD)	60	0.212 (pass)	0.402 (pass)
	90	0.278 (pass)	0.327 (pass)
	180	0.161 (pass)	0.008 (fail)
Sale Frequency	60	0.811 (pass)	0.101 (pass)
	90	0.570 (pass)	0.343 (pass)
	180	0.567 (pass)	0.057 (pass)
Discount Depth (%)	60	0.535 (pass)	0.368 (pass)
	90	0.415 (pass)	0.656 (pass)
	180	0.357 (pass)	0.165 (pass)

H0: all pre-period event-study coefficients = 0 jointly (Wald chi-square, reference week = -1). High p-values support parallel trends. 17 of 18 tests pass.

```

fail_row <- which(pretrend_wide$Outcome == "Price (USD)" & pretrend_wide$Window == 180)
ft_pre <- color(ft_pre, i = fail_row, j = "Synthetic Control DiD", color = "red")

save_as_docx(ft_pre, path = "Output/tab_pretrends.docx")

```

6.3 Minimum Detectable Effects

This converts each null into a bounded null. The minimum detectable effect is the smallest effect the design would detect with 80% power at conventional significance, reported both in levels and as a share of the pre-period mean. Read alongside the confidence interval bounds, it establishes which effect sizes the data rules out rather than merely failing to confirm.

```

pre_means <- steam_panel %>%
  filter(ever_treated, days_since_entry < 0, days_since_entry >= -max(window)) %>%
  summarise(across(all_of(outcome_cols), ~ mean(.x, na.rm = TRUE))) %>%
  pivot_longer(everything(), names_to = "outcome", values_to = "pre_mean")

mde <- comparison %>%

```

```

left_join(pre_means, by = "outcome") %>%
mutate(
  mde_80      = 2.8 * std.error,                # (1.96 + 0.84) * SE
  mde_pct     = round(100 * mde_80 / pre_mean, 1),
  ci_pct_lo   = round(100 * ci_lo / pre_mean, 1),
  ci_pct_hi   = round(100 * ci_hi / pre_mean, 1)
) %>%
select(label, window, method, estimate, pre_mean,
       mde_80, mde_pct, ci_pct_lo, ci_pct_hi) %>%
mutate(across(c(estimate, pre_mean, mde_80), ~ round(.x, 3)))

mde %>% print(n = 100)

## # A tibble: 24 x 9
##   label      window method estimate pre_mean mde_80 mde_pct ci_pct_lo ci_pct_hi
##   <chr>      <dbl> <chr>    <dbl>    <dbl>  <dbl>  <dbl>    <dbl>    <dbl>
## 1 Discount ~    60 Stack~  -0.119   10.7    1.69   15.8   -12.1     9.9
## 2 Discount ~    60 Synth~   0.5     10.7    1.57   14.6    -5.6    14.9
## 3 Discount ~    90 Stack~  -0.048   10.7    1.54   14.3   -10.5     9.6
## 4 Discount ~    90 Synth~   0.349   10.7    1.33   12.4    -5.4    11.9
## 5 Discount ~   180 Stack~   0.356   10.7    1.32   12.3    -5.3    11.9
## 6 Discount ~   180 Synth~   1.19    10.7    1.05    9.8     4.2    17.9
## 7 Condition~    60 Stack~  -0.908   55.3    2.20    4     -4.4     1.1
## 8 Condition~    60 Synth~   1.79    55.3    2.27    4.1     0.4     6.1
## 9 Condition~    90 Stack~   0.572   55.3    1.94    3.5    -1.4     3.5
## 10 Condition~   90 Synth~   1.29    55.3    2.19    4     -0.4     5.1
## 11 Condition~   180 Stack~   0.837   55.3    1.71    3.1    -0.6     3.7
## 12 Condition~   180 Synth~   2.25    55.3    1.93    3.5     1.6     6.5
## 13 Price (US~    60 Stack~   0.499   25.7    0.775    3     -0.2     4.1
## 14 Price (US~    60 Synth~   0.348   25.7    0.694    2.7    -0.5     3.2
## 15 Price (US~    90 Stack~   0.475   25.7    0.646    2.5     0.1     3.6
## 16 Price (US~    90 Synth~   0.53    25.7    0.58    2.3     0.5     3.6
## 17 Price (US~   180 Stack~   0.138   25.7    0.558    2.2    -1      2.1
## 18 Price (US~   180 Synth~   0.233   25.7    0.55    2.1    -0.6     2.4
## 19 Sale Freq~    60 Stack~  -0.006   0.194   0.03   15.3   -14      7.4
## 20 Sale Freq~    60 Synth~   0.009   0.194   0.033   16.7   -7.2    16.3
## 21 Sale Freq~    90 Stack~  -0.002   0.194   0.025   12.9   -9.9     8.1
## 22 Sale Freq~    90 Synth~   0.009   0.194   0.026   13.4   -4.7     14
## 23 Sale Freq~   180 Stack~   0.005   0.194   0.021   11.1   -5.4    10.1
## 24 Sale Freq~   180 Synth~   0.018   0.194   0.028   14.3   -0.9    19.1

```

6.4 Event Studies

```

dir.create("Output/es", recursive = TRUE, showWarnings = FALSE)

plot_es <- function(mod, ptrend, title, fname) {
  png(file.path("Output/es", fname), width = 8, height = 5, units = "in", res = 220)
  iplot(mod, ref.line = 0, xlab = "Weeks since Epic entry", main = title)
  mtext(sprintf("Joint pre-trend test: chi2(%d) = %.1f, p = %.3f",
               ptrend$df, ptrend$chi2, ptrend$p_value),
        side = 3, line = 0.2, cex = 0.8)
  dev.off()
}

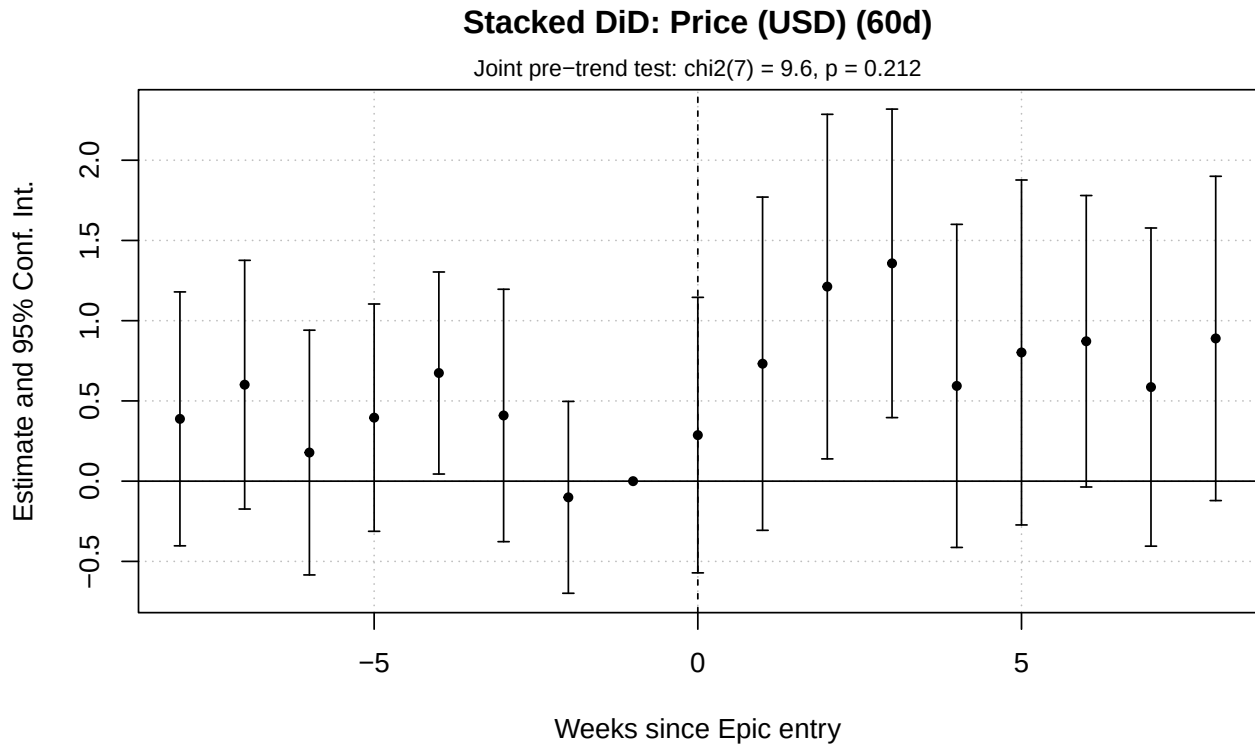
```

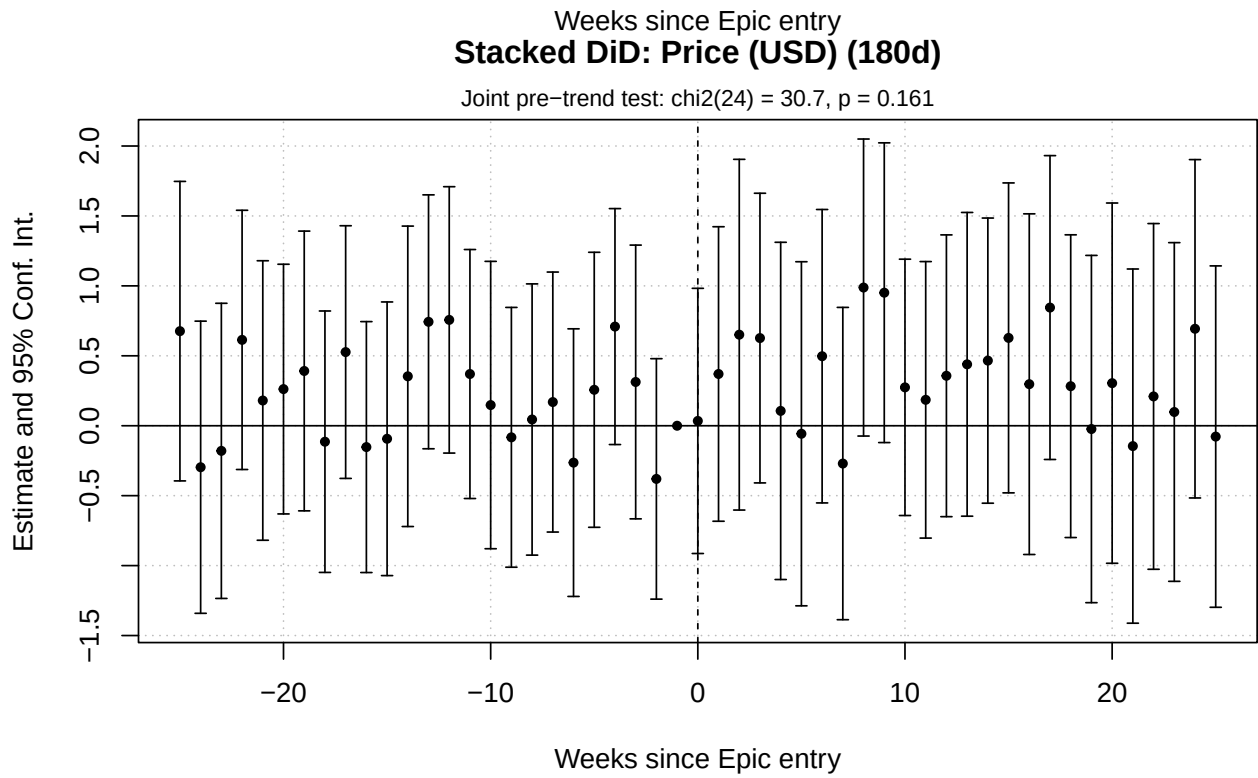
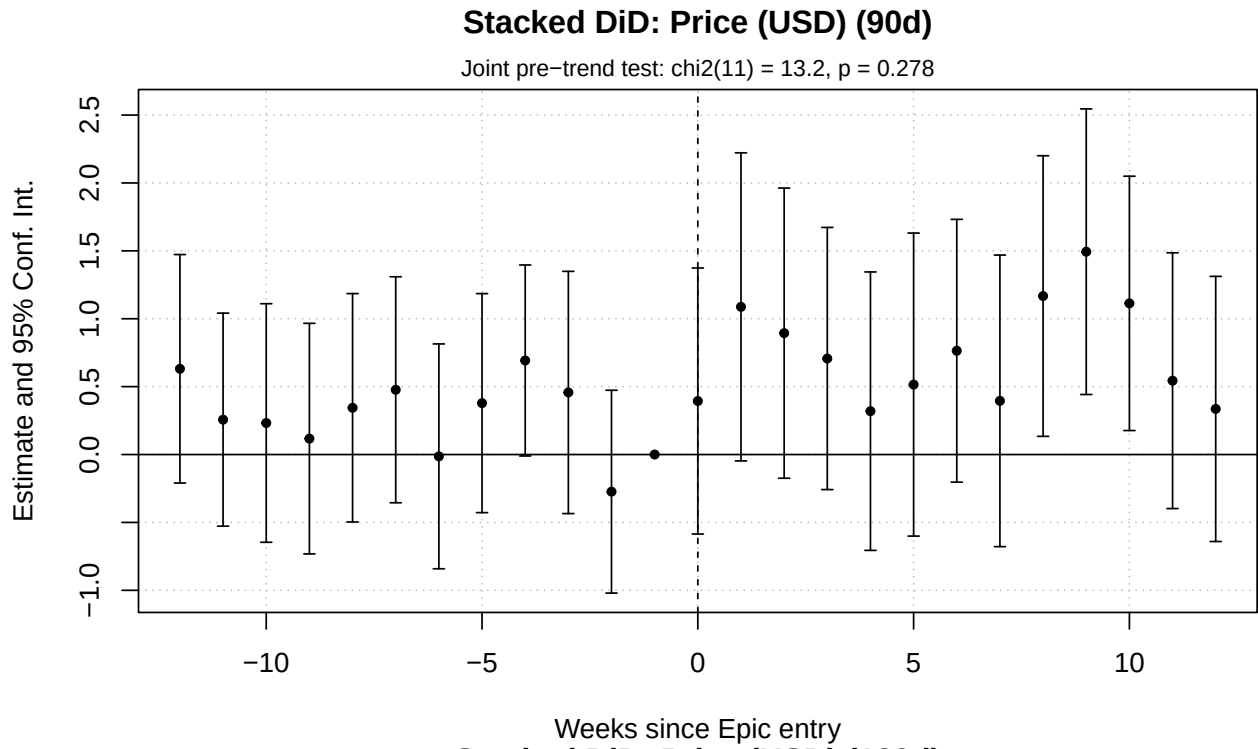
```

iplot(mod, ref.line = 0, xlab = "Weeks since Epic entry", main = title)
mtext(sprintf("Joint pre-trend test: chi2(%d) = %.1f, p = %.3f",
              ptrend$df, ptrend$chi2, ptrend$p_value),
       side = 3, line = 0.2, cex = 0.8)
}

for (oc in outcome_cols) {
  lab <- outcome_spec$label[outcome_spec$col == oc]
  for (nm in names(did_results)) {
    r <- did_results[[nm]]
    plot_es(r$models[[oc]]$es, r$models[[oc]]$pretrend,
            paste0("Stacked DiD: ", lab, " (", r$window, "d)"),
            paste0("stacked_", oc, "_", r$window, ".png"))
  }
  for (nm in names(synth_results)) {
    r <- synth_results[[nm]]
    x <- r$by_outcome[[oc]]
    if (is.null(x)) next
    plot_es(x$models$es, x$models$pretrend,
            paste0("Synthetic Control DiD: ", lab, " (", r$window, "d)"),
            paste0("synth_", oc, "_", r$window, ".png"))
  }
}

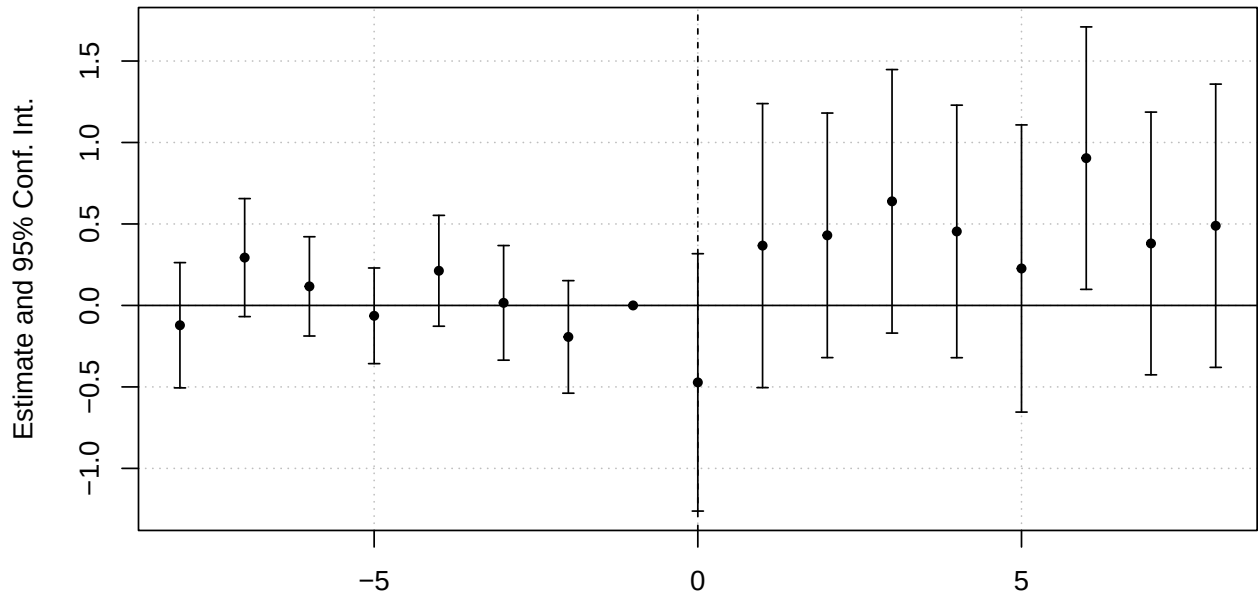
```





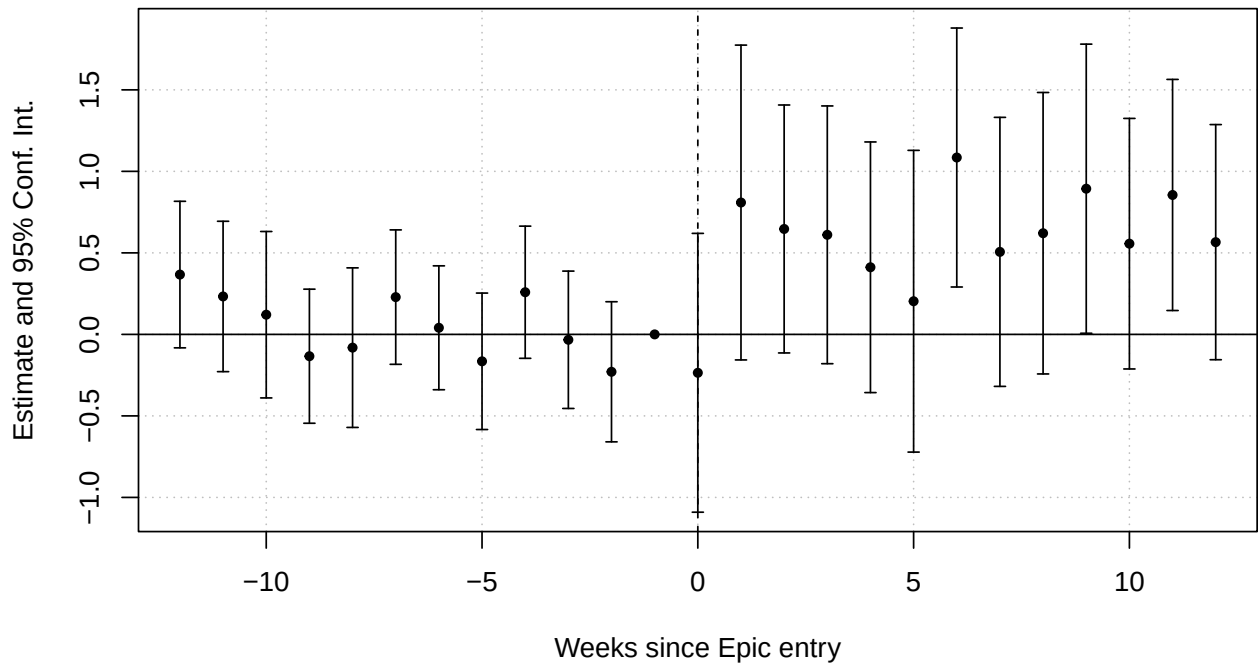
Synthetic Control DiD: Price (USD) (60d)

Joint pre-trend test: $\chi^2(7) = 7.3$, $p = 0.402$



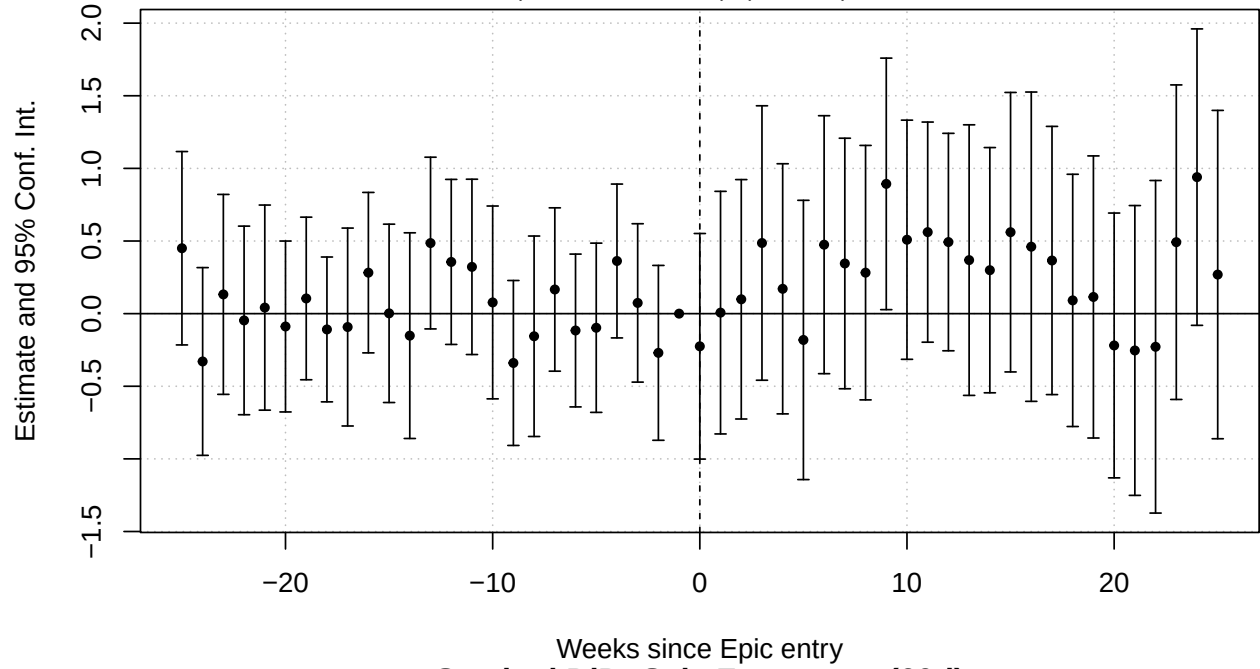
Synthetic Control DiD: Price (USD) (90d)

Joint pre-trend test: $\chi^2(11) = 12.5$, $p = 0.327$



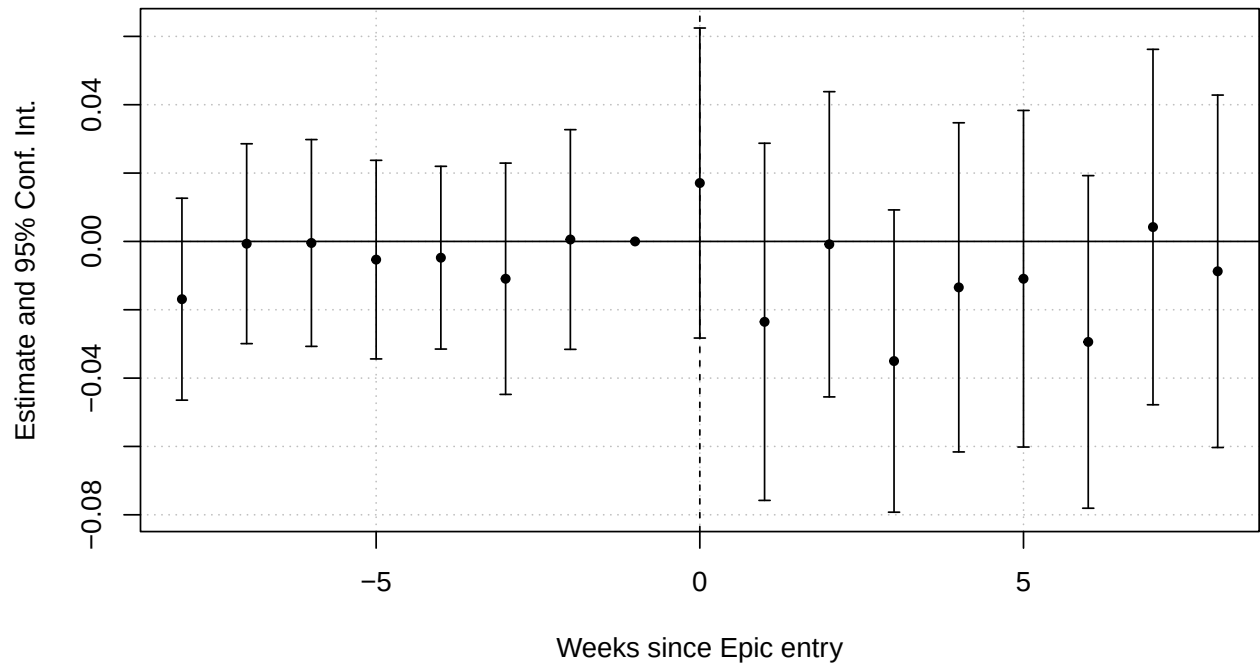
Synthetic Control DiD: Price (USD) (180d)

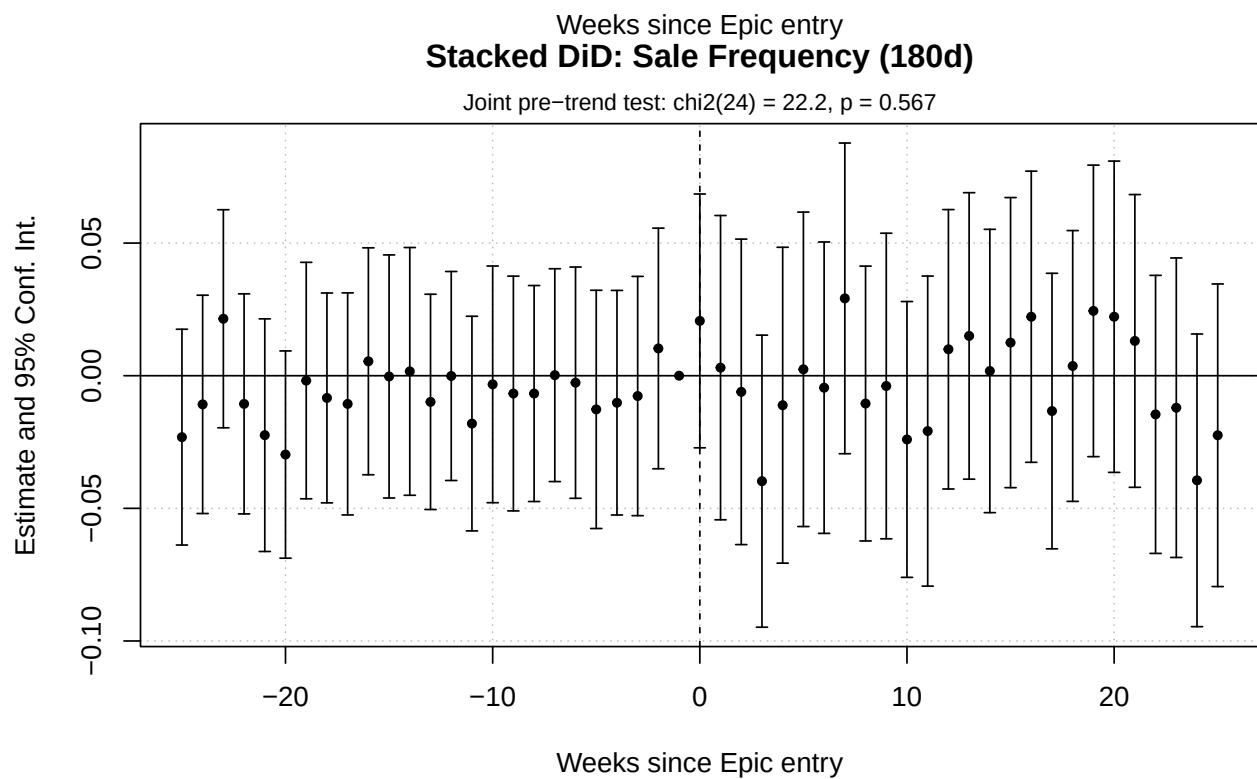
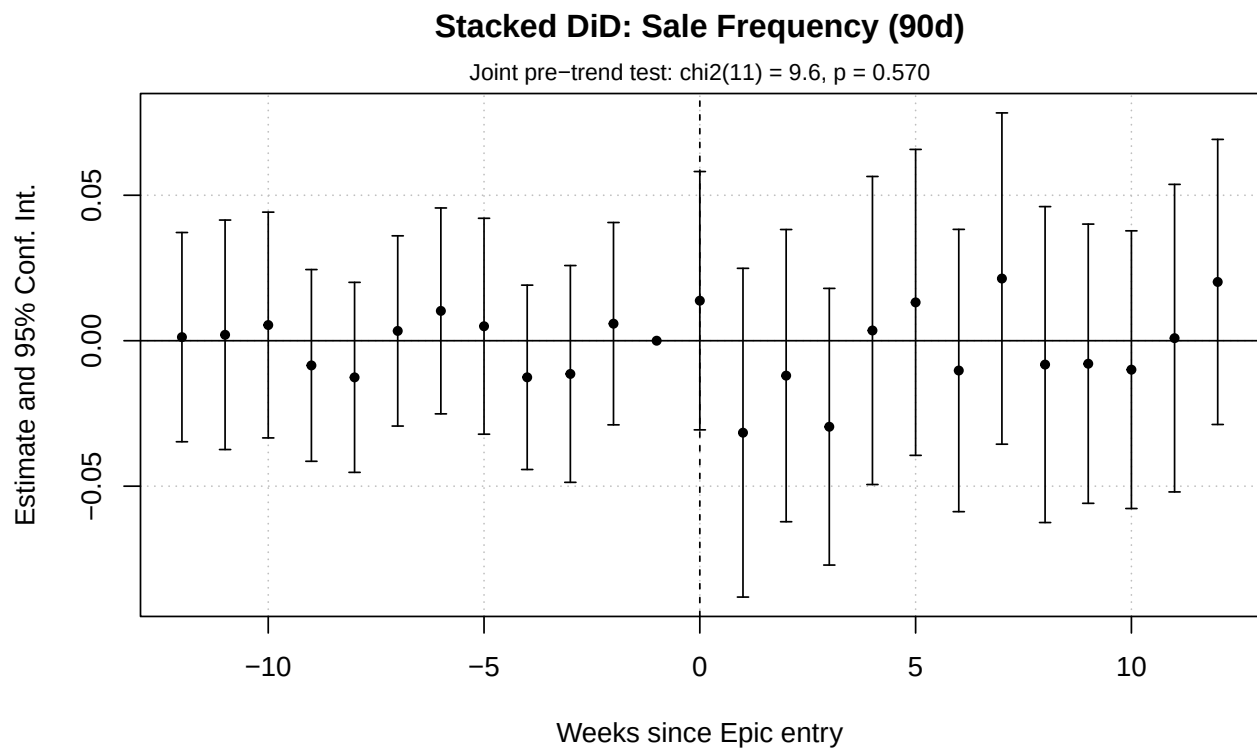
Joint pre-trend test: $\chi^2(24) = 43.7$, $p = 0.008$



Stacked DiD: Sale Frequency (60d)

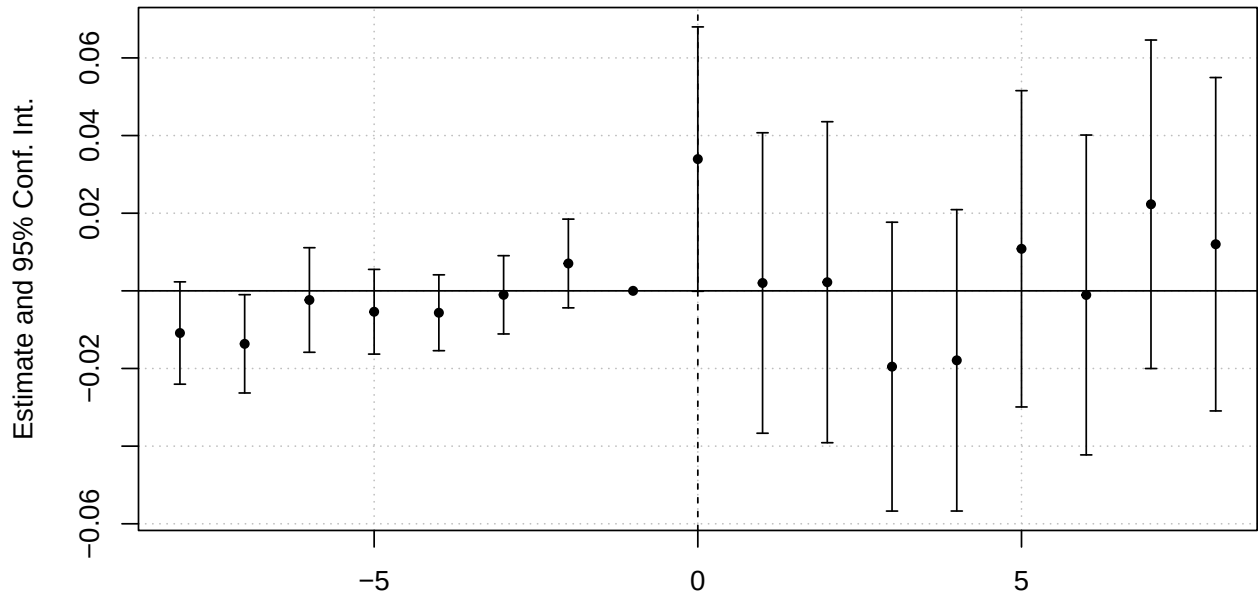
Joint pre-trend test: $\chi^2(7) = 3.7$, $p = 0.811$





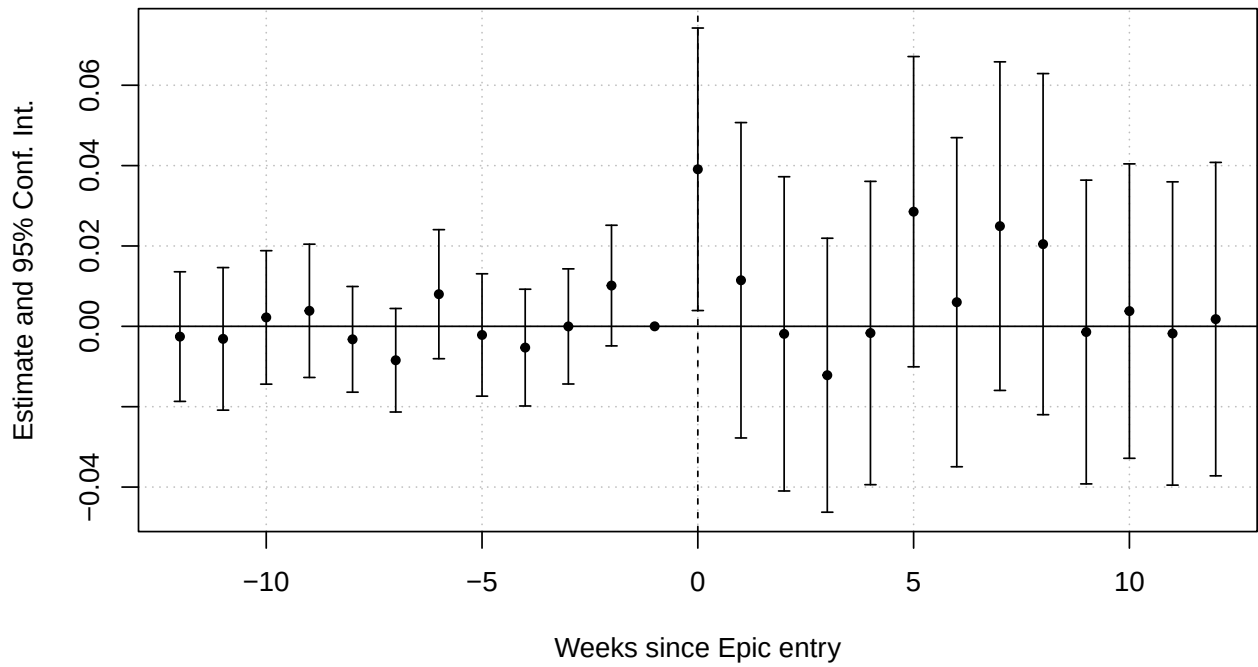
Synthetic Control DiD: Sale Frequency (60d)

Joint pre-trend test: $\chi^2(7) = 12.0$, $p = 0.101$



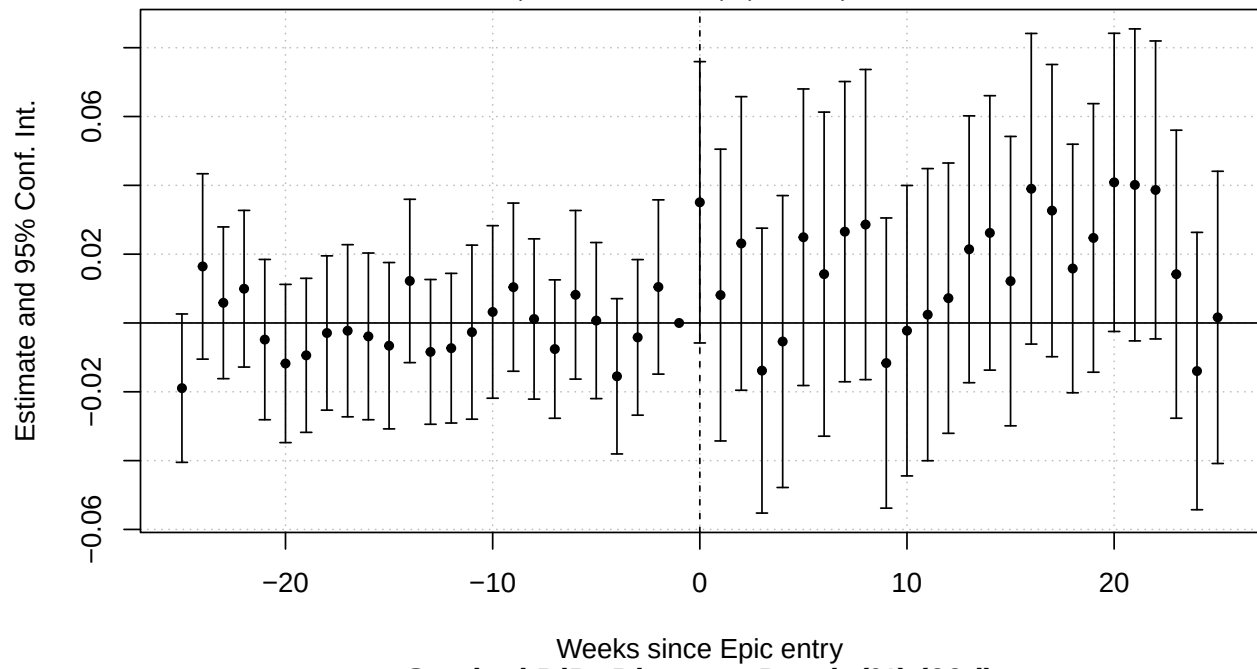
Synthetic Control DiD: Sale Frequency (90d)

Joint pre-trend test: $\chi^2(11) = 12.3$, $p = 0.343$



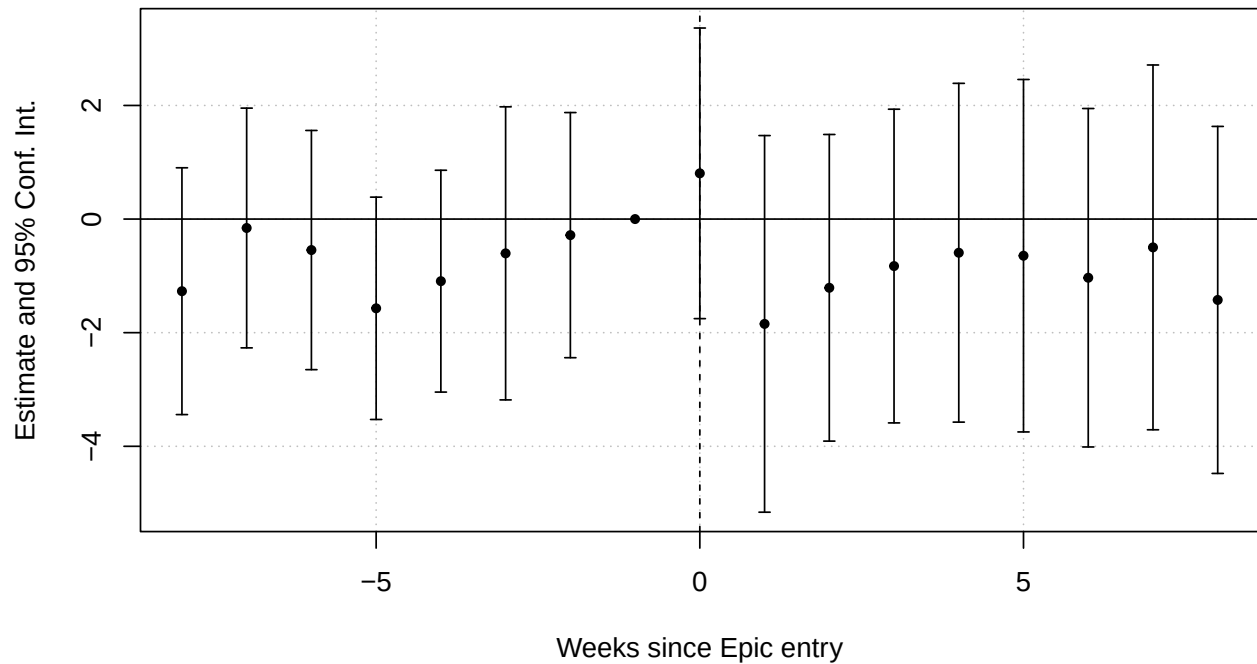
Synthetic Control DiD: Sale Frequency (180d)

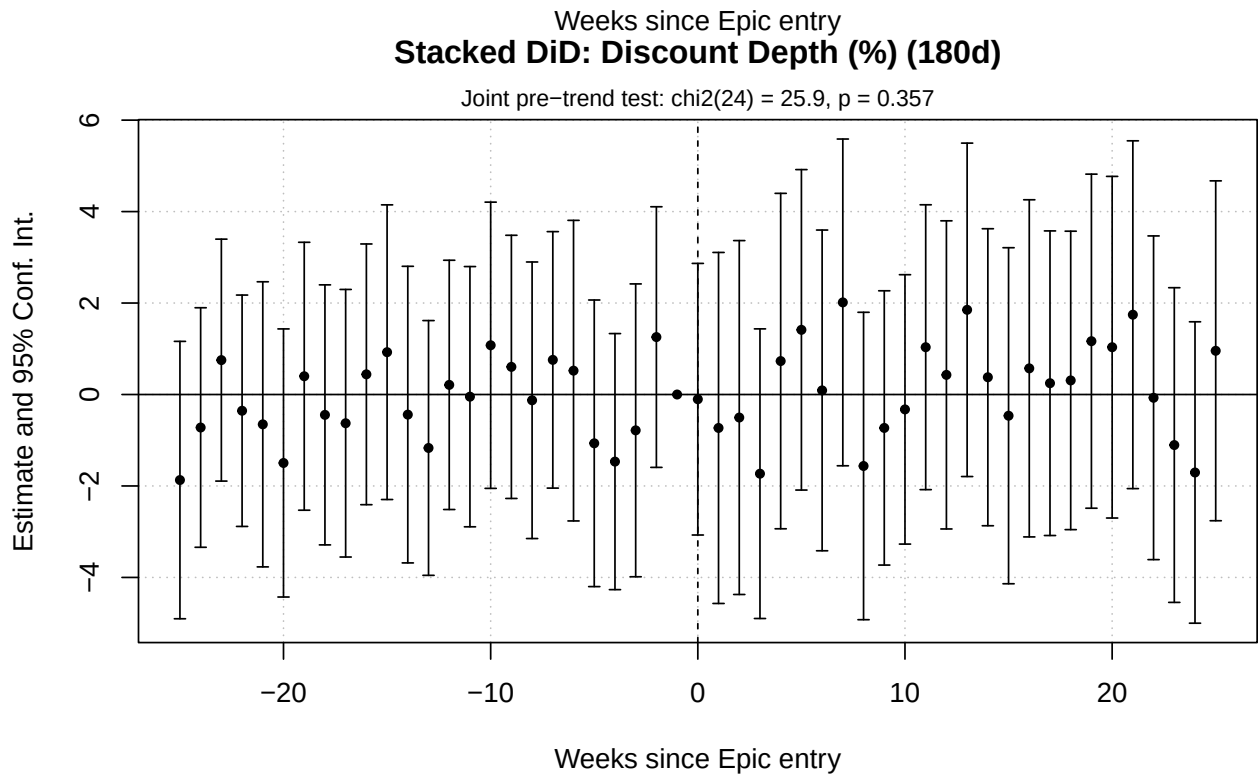
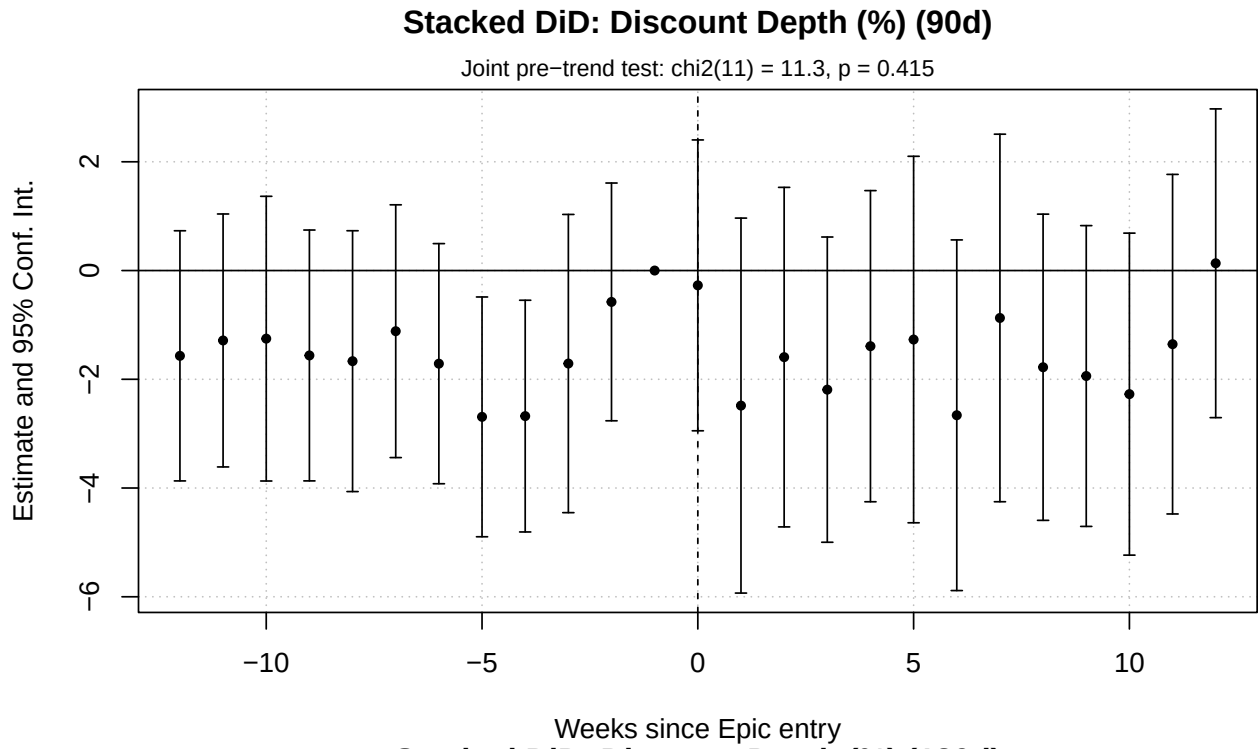
Joint pre-trend test: $\chi^2(24) = 35.9$, $p = 0.057$



Stacked DiD: Discount Depth (%) (60d)

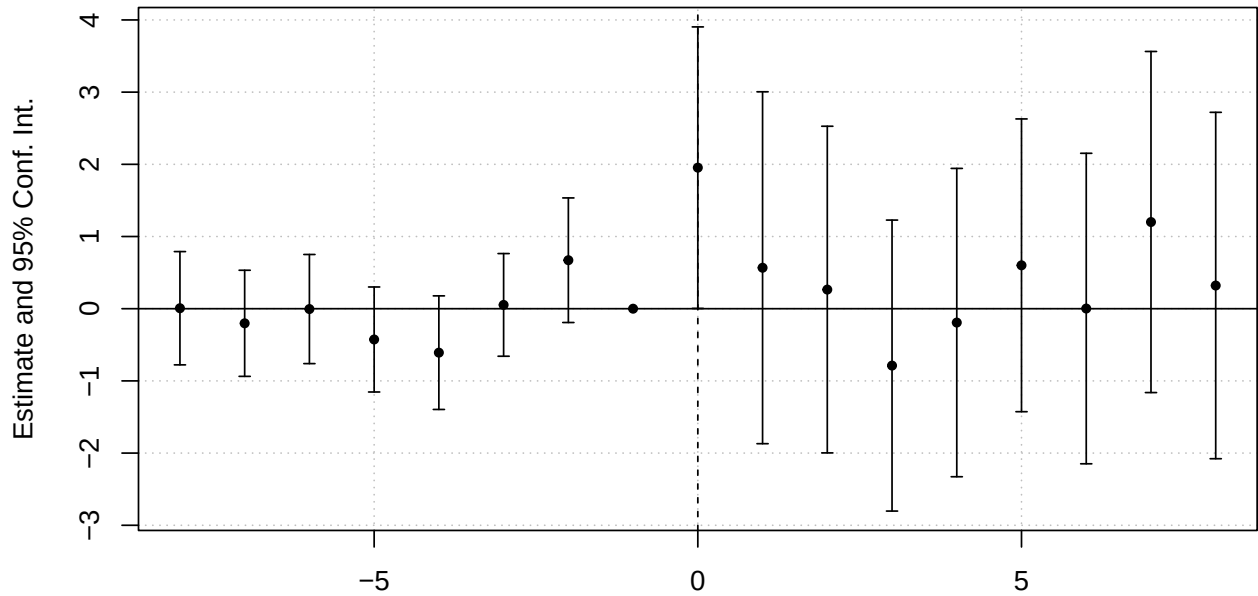
Joint pre-trend test: $\chi^2(7) = 6.0$, $p = 0.535$





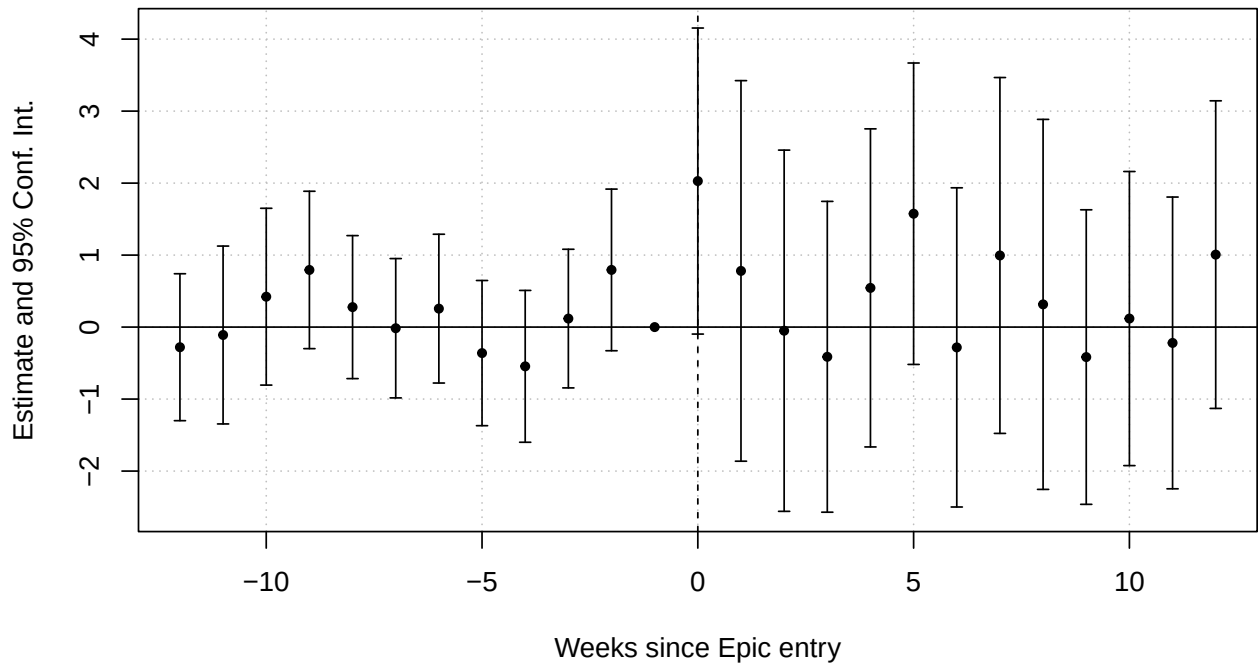
Synthetic Control DiD: Discount Depth (%) (60d)

Joint pre-trend test: $\chi^2(7) = 7.6$, $p = 0.368$



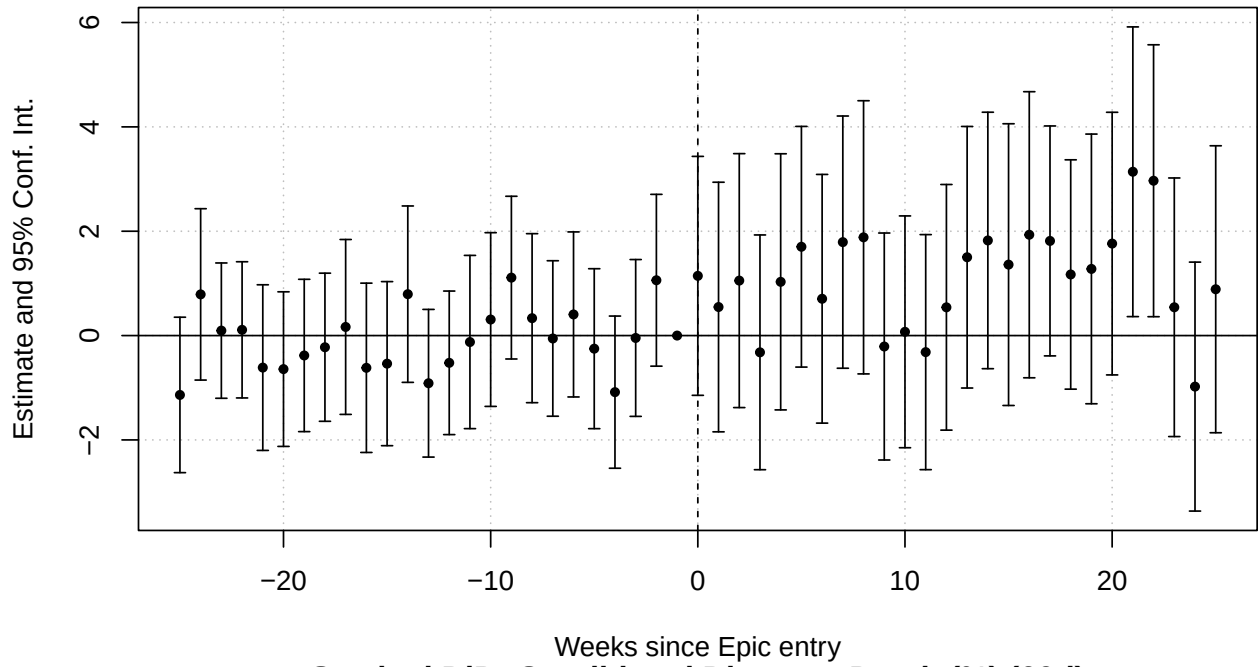
Synthetic Control DiD: Discount Depth (%) (90d)

Joint pre-trend test: $\chi^2(11) = 8.6$, $p = 0.656$



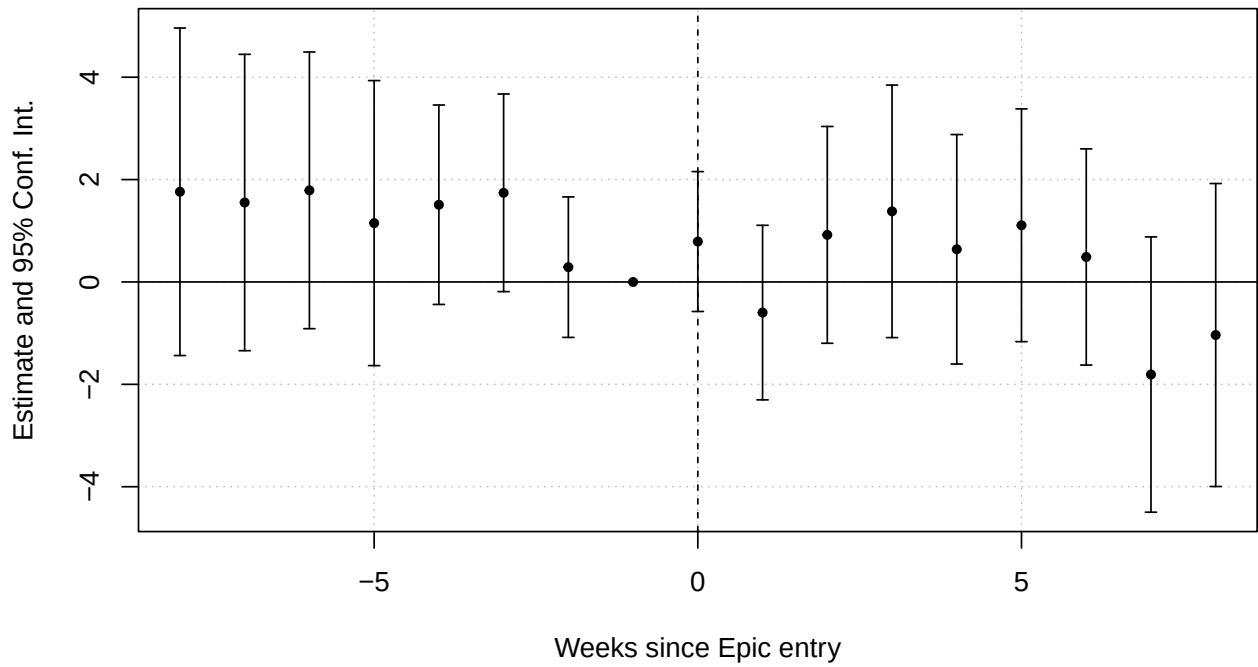
Synthetic Control DiD: Discount Depth (%) (180d)

Joint pre-trend test: $\chi^2(24) = 30.6$, $p = 0.165$



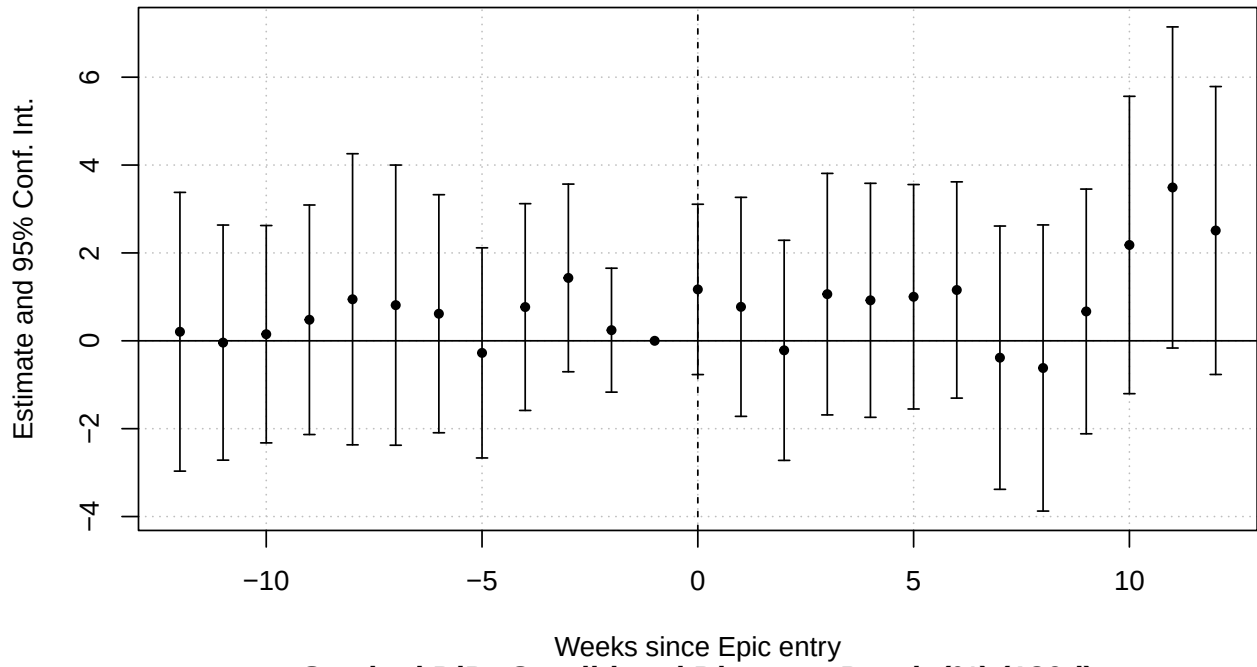
Stacked DiD: Conditional Discount Depth (%) (60d)

Joint pre-trend test: $\chi^2(7) = 6.9$, $p = 0.444$



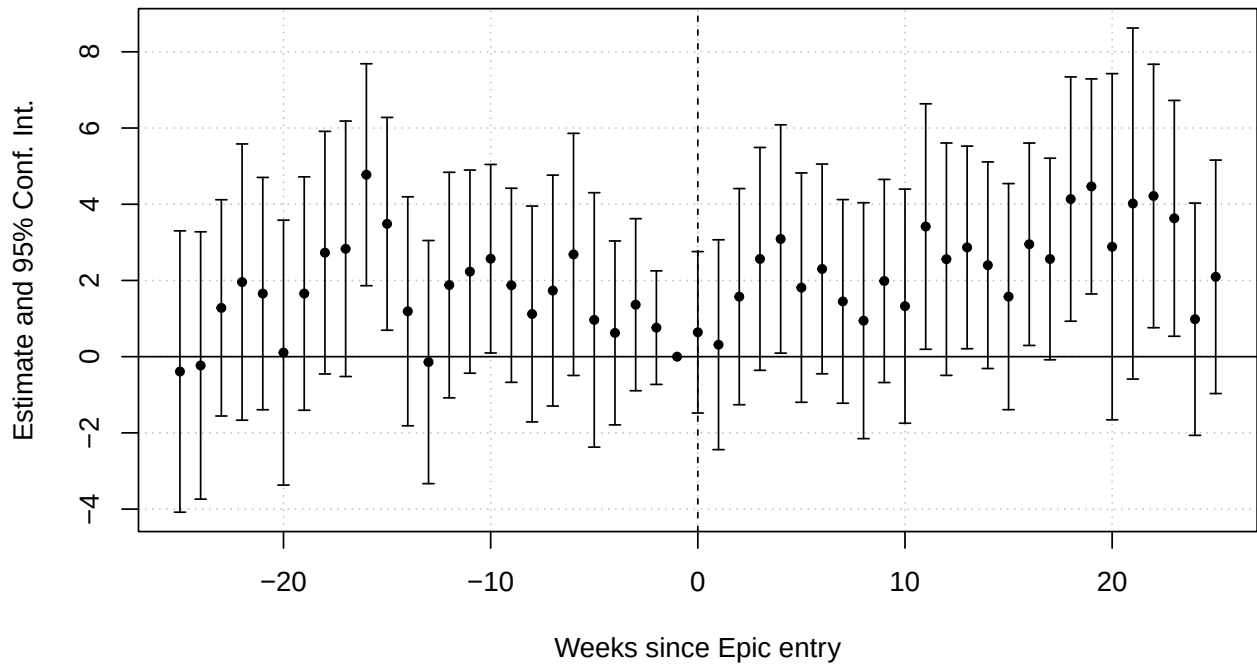
Stacked DiD: Conditional Discount Depth (%) (90d)

Joint pre-trend test: $\chi^2(11) = 6.8$, $p = 0.812$



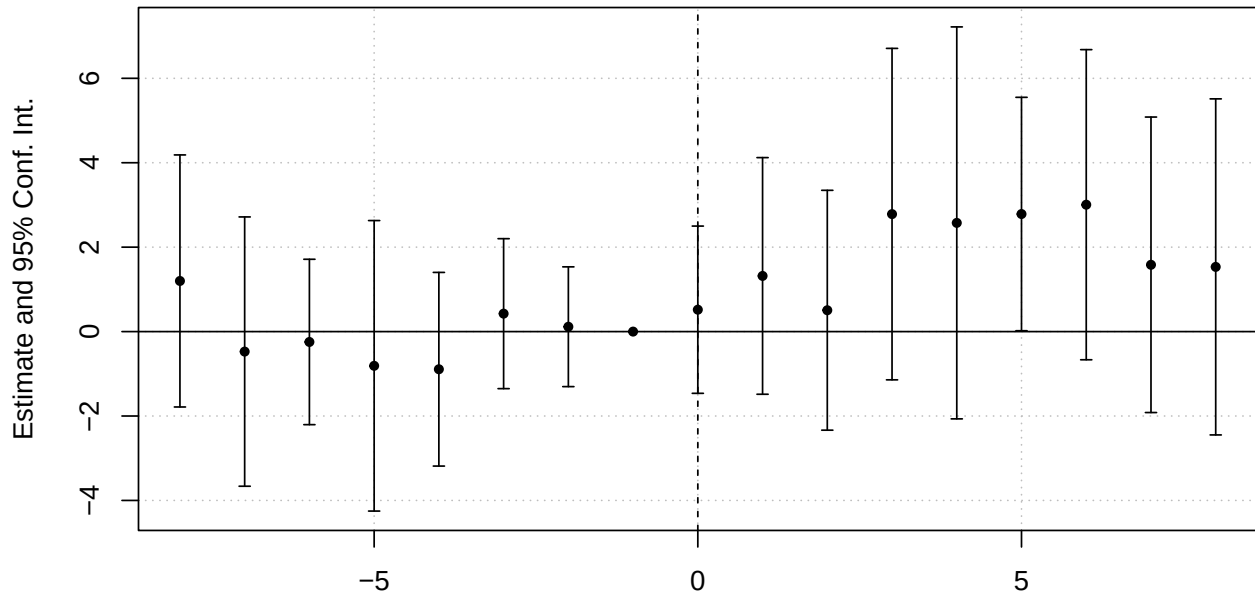
Stacked DiD: Conditional Discount Depth (%) (180d)

Joint pre-trend test: $\chi^2(24) = 26.6$, $p = 0.324$



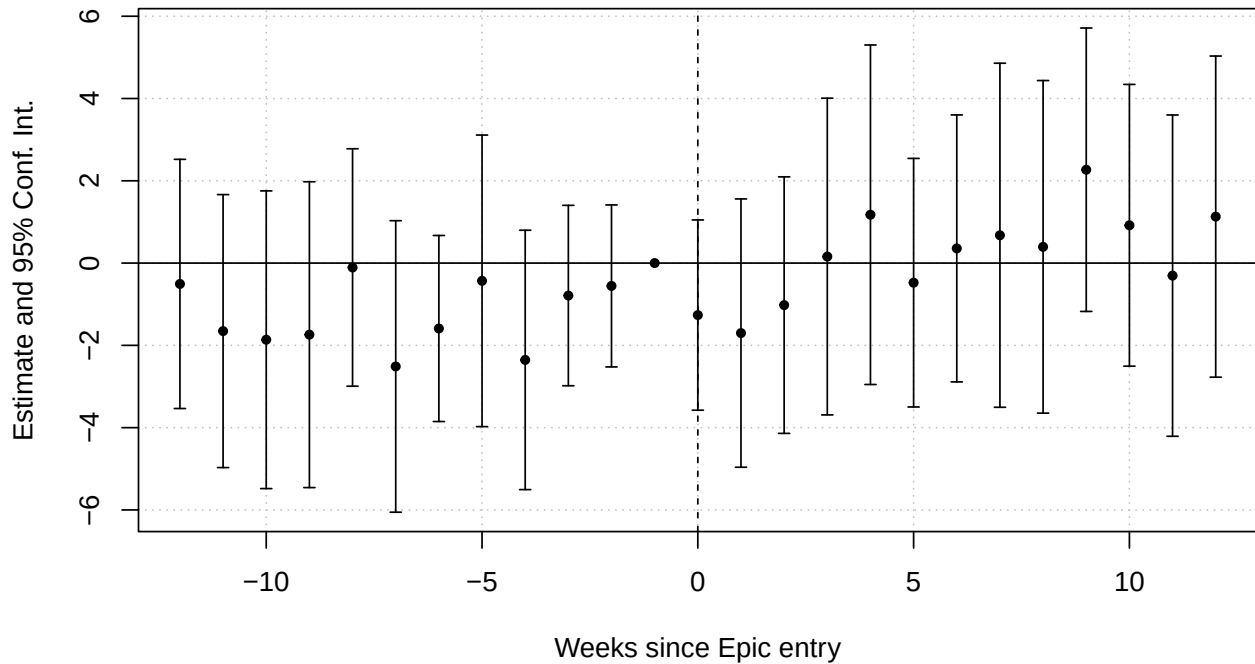
Synthetic Control DiD: Conditional Discount Depth (%) (60d)

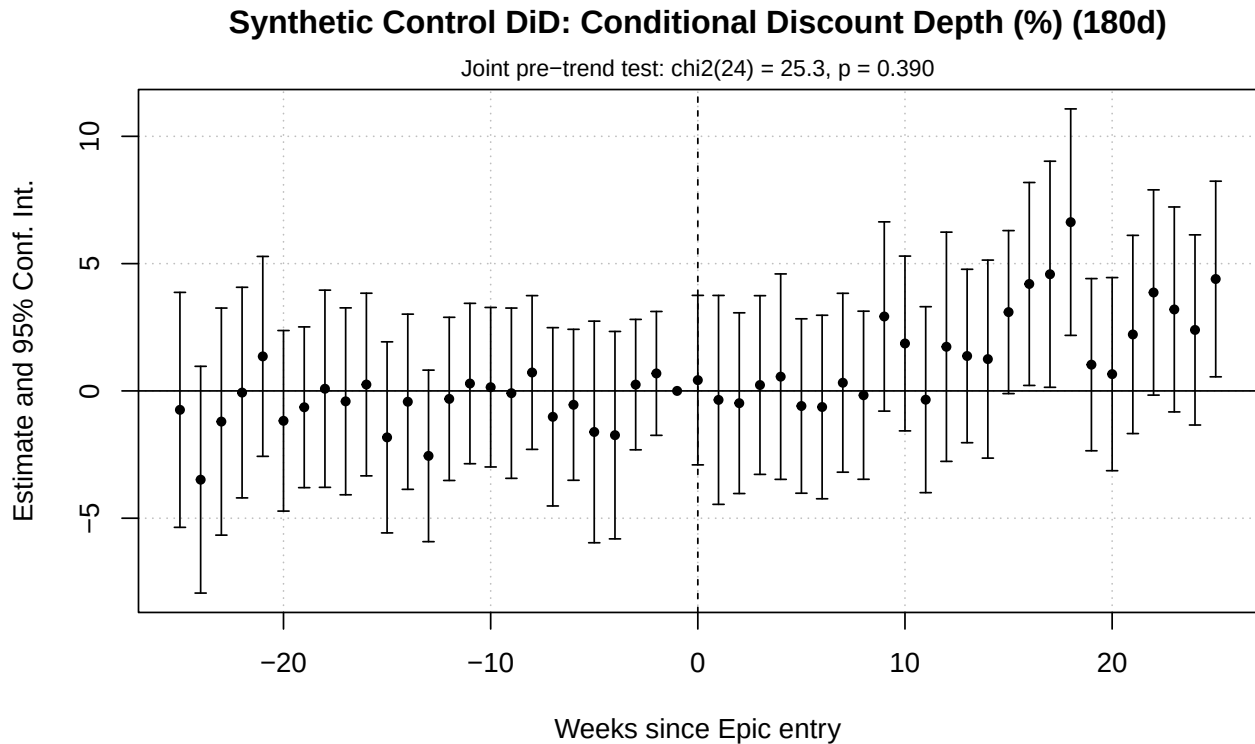
Joint pre-trend test: $\chi^2(7) = 4.4$, $p = 0.734$



Synthetic Control DiD: Conditional Discount Depth (%) (90d)

Joint pre-trend test: $\chi^2(11) = 10.0$, $p = 0.526$





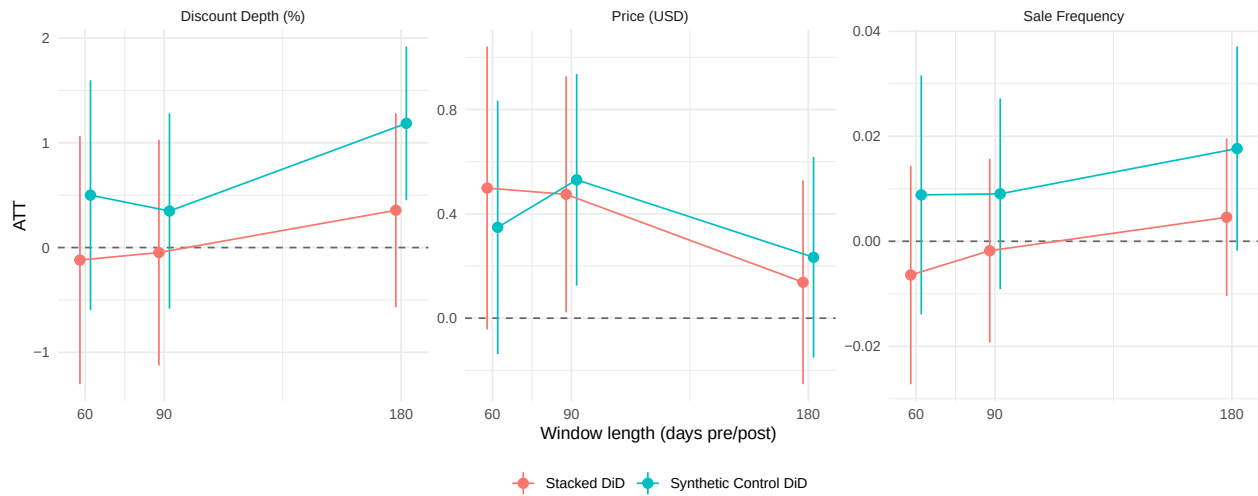
6.5 Consolidated Results Plot

```
p_all <- comparison %>%
  filter(outcome %in% outcome_spec$col[outcome_spec$primary]) %>%
  ggplot(aes(x = window, y = estimate, color = method)) +
  geom_hline(yintercept = 0, linetype = "dashed", color = "gray40") +
  geom_pointrange(aes(ymin = ci_lo, ymax = ci_hi),
    position = position_dodge(width = 8)) +
  geom_line(position = position_dodge(width = 8)) +
  facet_wrap(~ label, scales = "free_y") +
  scale_x_continuous(breaks = windows) +
  labs(title = "Effect of Epic Listing on Steam Outcomes: Two Estimators",
    subtitle = "Pooled treated x post ATT with 95% CI",
    x = "Window length (days pre/post)", y = "ATT", color = NULL) +
  theme_minimal() + theme(legend.position = "bottom")

p_all
```

Effect of Epic Listing on Steam Outcomes: Two Estimators

Pooled treated x post ATT with 95% CI



```
ggsave("Output/es/att_comparison.png", p_all, width = 11, height = 5, dpi = 220)
```

7. Placebo Tests

7.1 In-Time Placebo

Entry is shifted back one full window, so everything inside the fake window is genuinely pre-treatment and the estimated effect should be zero.

```
placebo_intime <- map_dfr(windows, function(w) {
  n_pre <- floor(w / 7)
  fake_entry <- entry_lookup - w # shift back one full window

  map_dfr(outcome_spec$col[outcome_spec$primary], function(oc) {
    m <- match_controls(w, n_pre, oc, entry_map = fake_entry)
    if (is.null(m) || nrow(m) == 0) return(NULL)
    d <- build_stacked(m, w, entry_map = fake_entry)
    if (is.null(d) || nrow(d) == 0) return(NULL)
    fit <- feols(as.formula(paste0(oc, " ~ treated:post | subexp^game_title + subexp^event_week")),
      data = d, cluster = ~ game_title)
    tidy(fit) %>% filter(term == "treated:post") %>%
      transmute(window = w, outcome = oc, estimate, std.error, p.value,
        n_treated = nrow(m))
  })
}) %>%
  left_join(outcome_spec %>% select(outcome = col, label), by = "outcome") %>%
  mutate(across(c(estimate, std.error), ~ round(.x, 3)),
    p.value = round(p.value, 3),
    flag = if_else(p.value < .05, "FAILS PLACEBO", "ok"))

placebo_intime
```

```
## # A tibble: 9 x 8
##   window outcome estimate std.error p.value n_treated label      flag
##   <dbl> <chr>      <dbl>      <dbl>   <dbl>   <int> <chr> <chr>
```

## 1	60 price	0.073	0.29	0.802	238 Price (USD)	ok
## 2	60 sale	0.006	0.01	0.532	238 Sale Frequency	ok
## 3	60 disc	-0.303	0.617	0.624	238 Discount Depth (%)	ok
## 4	90 price	-0.078	0.218	0.719	232 Price (USD)	ok
## 5	90 sale	0.013	0.009	0.138	232 Sale Frequency	ok
## 6	90 disc	0.603	0.599	0.315	232 Discount Depth (%)	ok
## 7	180 price	-0.302	0.216	0.163	212 Price (USD)	ok
## 8	180 sale	-0.007	0.006	0.278	212 Sale Frequency	ok
## 9	180 disc	0.221	0.377	0.558	212 Discount Depth (%)	ok

7.2 Permutation Placebo

The treated label is randomly reassigned within each matched pair and the model re-estimated 500 times. Under the null the true estimate should sit in the middle of this distribution. The resulting p-value makes no distributional assumptions, so it serves as a check on the cluster-robust p-values reported above. This is run at the 90-day window, where the one significant price estimate appears.

```
set.seed(20260630)
N_PERM <- 500

perm_null <- map_dfr(outcome_spec$col[outcome_spec$primary], function(oc) {
  key <- outcome_spec$match_on[outcome_spec$col == oc]
  d <- did_results$w90$stacked[[key]]
  obs <- coef(did_results$w90$models[[oc]]$att)[["treated:post"]]

  draws <- map_dbl(seq_len(N_PERM), function(i) {
    flip <- tibble(subexp = unique(d$subexp),
                  swap = sample(c(TRUE, FALSE), n_distinct(d$subexp), replace = TRUE))
    dp <- d %>% left_join(flip, by = "subexp") %>%
      mutate(treated = if_else(swap, 1L - treated, treated))
    coef(feols(as.formula(paste0(oc, " ~ treated:post | subexp^game_title + subexp^event_week")),
              data = dp, cluster = ~ game_title))["treated:post"]
  })

  tibble(outcome = oc,
         observed = obs,
         perm_mean = mean(draws),
         perm_sd = sd(draws),
         pct_more_extreme = mean(abs(draws) >= abs(obs)),
         ri_p = mean(abs(draws) >= abs(obs)))
}) %>%
  left_join(outcome_spec %>% select(outcome = col, label), by = "outcome") %>%
  mutate(across(where(is.numeric), ~ round(.x, 4)))

perm_null
```

```
## # A tibble: 3 x 7
##   outcome observed perm_mean perm_sd pct_more_extreme ri_p label
##   <chr>      <dbl>    <dbl>    <dbl>          <dbl> <dbl> <chr>
## 1 price      0.475    -0.0031  0.236          0.036 0.036 Price (USD)
## 2 sale     -0.0018     0.0003  0.009          0.842 0.842 Sale Frequency
## 3 disc     -0.0479    -0.0131  0.535          0.942 0.942 Discount Depth (%)
```

8. Output Tables

Word versions of the result tables for the thesis, plus the diagnostic tables that went into the appendix and the defense backup slides.

```
library(flextable)
dir.create("Output", showWarnings = FALSE)

att_stacked <- map(setNames(outcome_cols, outcome_cols), function(oc)
  map(did_results, ~ .x$models[[oc]]$att))
att_synth <- map(setNames(outcome_cols, outcome_cols), function(oc)
  map(synth_results, function(r) {
    x <- r$by_outcome[[oc]]; if (is.null(x)) NULL else x$models$att
  }) %>% compact())

make_did_table <- function(oc, oc_label) {
  stk <- att_stacked[[oc]]
  syn <- att_synth[[oc]]

  # Window labels taken from each list separately, in case a synth model failed
  # for some window (then the header still lines up).
  stk_lab <- paste0(sub("^w", "", names(stk)), "d")
  syn_lab <- paste0(sub("^w", "", names(syn)), "d")

  names(stk) <- paste0("S_", stk_lab)
  names(syn) <- paste0("Y_", syn_lab)
  mods <- c(stk, syn)

  ft <- modelsummary(
    mods,
    coef_map = c("treated:post" = "ATT (treated x post)"),
    gof_omit = "R2|AIC|BIC|Log\\.Lik\\.|RMSE|Std\\.Errors|FE:",
    stars = c('*' = .05, '**' = .01, '***' = .001),
    title = paste0("Effect of Epic Listing on ", oc_label,
      ": Stacked vs. Synthetic Control DiD"),
    output = "flextable"
  )

  mk <- ft$col_keys[-1]
  ft <- set_header_labels(ft,
    values = setNames(as.list(c(stk_lab, syn_lab)), mk))
  ft <- add_header_row(ft,
    values = c("", "Stacked DiD", "Synthetic Control DiD"),
    colwidths = c(1, length(stk), length(syn)), top = TRUE)

  ft %>% theme_booktabs() %>% align(align = "center", part = "header") %>% autofit()
}

did_price_ft <- make_did_table("price", "Price (USD)")
did_sale_ft <- make_did_table("sale", "Sale Frequency")
did_disc_ft <- make_did_table("disc", "Discount Depth (%)")
did_cond_ft <- make_did_table("disc_cond", "Conditional Discount Depth (%)")

did_price_ft; did_sale_ft; did_disc_ft; did_cond_ft
```

Table 2: Effect of Epic Listing on Price (USD): Stacked vs. Synthetic Control DiD

	Stacked DiD			Synthetic Control DiD		
	60d	90d	180d	60d	90d	180d
ATT (treated x post)	0.499 (0.277)	0.475* (0.231)	0.138 (0.199)	0.348 (0.248)	0.530* (0.207)	0.233 (0.196)
Num.Obs.	8596	12110	22986	8586	12104	22974
* p < 0.05, ** p < 0.01, *** p < 0.001						

Table 3: Effect of Epic Listing on Sale Frequency: Stacked vs. Synthetic Control DiD

	Stacked DiD			Synthetic Control DiD		
	60d	90d	180d	60d	90d	180d
ATT (treated x post)	-0.006 (0.011)	-0.002 (0.009)	0.005 (0.008)	0.009 (0.012)	0.009 (0.009)	0.018 (0.010)
Num.Obs.	8596	12110	22986	8586	12104	22974
* p < 0.05, ** p < 0.01, *** p < 0.001						

Table 4: Effect of Epic Listing on Discount Depth (%): Stacked vs. Synthetic Control DiD

	Stacked DiD			Synthetic Control DiD		
	60d	90d	180d	60d	90d	180d
ATT (treated x post)	-0.119 (0.605)	-0.048 (0.549)	0.356 (0.472)	0.500 (0.560)	0.349 (0.476)	1.186** (0.374)
Num.Obs.	8596	12110	22986	8586	12104	22974
* p < 0.05, ** p < 0.01, *** p < 0.001						

Table 5: Effect of Epic Listing on Conditional Discount Depth (%): Stacked vs. Synthetic Control DiD

	Stacked DiD			Synthetic Control DiD		
	60d	90d	180d	60d	90d	180d
ATT (treated x post)	-0.908 (0.786)	0.572 (0.693)	0.837 (0.610)	1.794* (0.812)	1.289 (0.783)	2.252** (0.689)
Num.Obs.	1684	2390	4398	2036	3190	6666

Table 5: Effect of Epic Listing on Conditional Discount Depth (%):
Stacked vs. Synthetic Control DiD

	Stacked DiD			Synthetic Control DiD		
	60d	90d	180d	60d	90d	180d
* p < 0.05, ** p < 0.01, *** p < 0.001						

```
save_as_docx(did_price_ft, did_sale_ft, did_disc_ft, did_cond_ft,
             path = "Output/tab_did_att.docx")

# Diagnostic tables for the appendix / defence backup slides
save_as_docx(
  flextable(stacked_quality) %>% theme_booktabs() %>% autofit(),
  flextable(synth_quality) %>% theme_booktabs() %>% autofit(),
  flextable(fit_comparison) %>% theme_booktabs() %>% autofit(),
  flextable(pretrends) %>% theme_booktabs() %>% autofit(),
  flextable(mde) %>% theme_booktabs() %>% autofit(),
  flextable(reuse_table) %>% theme_booktabs() %>% autofit(),
  flextable(balance_table) %>% theme_booktabs() %>% autofit(),
  flextable(placebo_intime) %>% theme_booktabs() %>% autofit(),
  flextable(perm_null) %>% theme_booktabs() %>% autofit(),
  path = "Output/tab_did_diagnostics.docx"
)
```

Extra Information and Code

This section holds code that supported the analysis but does not appear in the thesis. It is kept because it documents decisions that would otherwise look arbitrary, particularly the choice of weight solver.

Verifying How Sale and Discount Depth Are Coded

This determines whether discount depth is a conditional or an unconditional measure. If discount percent is recorded as 0 on non-sale days rather than as missing, then the measure is unconditional and is mechanically tied to sale frequency. That is what this check confirmed, and it is the reason the conditional measure was added as a fourth outcome.

```
c13 <- data %>%
  filter(shop_name == "Steam") %>%
  summarise(
    n_rows = n(),
    sale_values = paste(sort(unique(sale)), collapse = ", "),
    n_disc_NA = sum(is.na(discount_percent)),
    # what does discount_percent look like when NOT on sale?
    disc_when_not_on_sale_mean = mean(discount_percent[sale == 0], na.rm = TRUE),
    disc_when_not_on_sale_max = suppressWarnings(max(discount_percent[sale == 0],
                                                       na.rm = TRUE)),
    disc_when_not_on_sale_pct_NA = mean(is.na(discount_percent[sale == 0])) * 100,
    # and when on sale?
    disc_when_on_sale_mean = mean(discount_percent[sale == 1], na.rm = TRUE),
    disc_when_on_sale_median = median(discount_percent[sale == 1], na.rm = TRUE)
```

```
)
c13

## # A tibble: 1 x 8
##   n_rows sale_values n_disc_NA disc_when_not_on_sale_m-1 disc_when_not_on_sal-2
##   <int> <chr>         <int>                <dbl>                <dbl>
## 1 2088825 0, 1           0                  0                  0
## # i abbreviated names: 1: disc_when_not_on_sale_mean,
## #   2: disc_when_not_on_sale_max
## # i 3 more variables: disc_when_not_on_sale_pct_NA <dbl>,
## #   disc_when_on_sale_mean <dbl>, disc_when_on_sale_median <dbl>
```

Sparsity of Conditional Discount Depth

Conditional depth is only defined in weeks where a sale actually occurred, and that condition is itself an outcome. The share of missing game-weeks below is why it is treated as exploratory rather than as a primary result.

```
tibble(
  pct_weeks_disc_cond_missing_treated = mean(is.na(treated_weekly_cal$disc_cond)) * 100,
  pct_weeks_disc_cond_missing_control = mean(is.na(control_weekly_cal$disc_cond)) * 100
)

## # A tibble: 1 x 2
##   pct_weeks_disc_cond_missing_treated pct_weeks_disc_cond_missing_control
##   <dbl>                                <dbl>
## 1      70.1                            72.4
```

Legacy Gradient-Descent Solver

The original synthetic control weights came from a hand-written projected gradient descent routine with a fixed learning rate and no convergence check. On price series at typical levels the gradient is large enough that a single update pushed every weight past zero, after which the clip-and-renormalise step returned something close to a uniform average of the donors. The function is kept here only so the comparison below can be reproduced.

```
solve_synth_weights_gd <- function(Y_pre, X_pre, n_iter = 500, lr = 0.01, trace = FALSE) {
  X_pre <- as.matrix(X_pre)
  J <- ncol(X_pre)
  w <- rep(1 / J, J)
  loss <- numeric(n_iter)
  for (it in seq_len(n_iter)) {
    resid <- as.vector(X_pre %*% w) - Y_pre
    loss[it] <- mean(resid^2)
    grad <- as.vector(t(X_pre) %*% resid) * (2 / length(Y_pre))
    w <- w - lr * grad
    w[w < 0] <- 0
    s <- sum(w)
    if (s > 0) w <- w / s else w <- rep(1 / J, J)
  }
  if (trace) attr(w, "loss") <- loss
  w
}
```

Solver Validation

The symptom that gave the bug away was that the synthetic control was fitting the pre-period worse than the single nearest-neighbour match. That should be impossible. A single donor corresponds to a vertex of the feasible simplex, so a correctly solved synthetic control can always fall back on it and can never do worse.

The comparison below runs the old solver, the new solver, and the best single donor on the same donor pool, on levels, so the optimiser is the only thing that differs. If the old weights are worse than the best single donor, the old solver is broken. The weight diagnostics show the same thing from the other direction: a maximum weight close to the minimum weight means the routine never moved away from its uniform starting point.

```
extract_donor_matrix <- function(tg, w, n_weeks, outcome = "price") {
  E <- entry_lookup[[tg]]
  entry_cal <- as.integer(floor(as.numeric(E - origin) / 7))
  pre_weeks <- (entry_cal - n_weeks):(entry_cal - 1)
  tr_genres <- treated_info %>% filter(game_title == tg) %>%
    select(all_of(genre_cols)) %>% unlist()

  t_pre <- treated_weekly_cal %>%
    filter(game_title == tg, cal_week %in% pre_weeks) %>%
    transmute(cal_week, y = .data[[outcome]]) %>%
    filter(!is.na(y)) %>% arrange(cal_week)
  if (nrow(t_pre) < 4) return(NULL)

  shared <- as.integer(control_genre_mat %*% tr_genres)
  elig <- control_ranges %>%
    mutate(shared_genres = shared) %>%
    filter(shared_genres >= 1, min_date <= E - w, max_date >= E + w) %>%
    pull(game_title)
  if (length(elig) < 3) return(NULL)

  wide <- control_weekly_cal %>%
    filter(game_title %in% elig, cal_week %in% t_pre$cal_week) %>%
    transmute(game_title, cal_week, value = .data[[outcome]]) %>%
    pivot_wider(names_from = game_title, values_from = value) %>%
    arrange(cal_week)

  X <- wide %>% select(-cal_week)
  keep <- colSums(is.na(X)) == 0
  if (sum(keep) < 3) return(NULL)
  X <- as.matrix(X[, keep, drop = FALSE])
  Y <- t_pre$y[match(wide$cal_week, t_pre$cal_week)]
  ok <- !is.na(Y)
  if (sum(ok) < 4) return(NULL)
  list(Y = Y[ok], X = X[ok, , drop = FALSE], game = tg)
}

rmse <- function(a, b) sqrt(mean((a - b)^2))

set.seed(1)
c1_games <- sample(treated_info$game_title, min(15, nrow(treated_info)))

c1 <- map_dfr(c1_games, function(tg) {
  d <- extract_donor_matrix(tg, 180, floor(180 / 7), "price")
```

```

if (is.null(d)) return(NULL)
Y <- d$Y; X <- d$X

w_gd <- solve_synth_weights_gd(Y, X)
w_qp <- solve_synth_weights(Y, X, demean = FALSE) # levels: apples-to-apples with GD
w_tr <- solve_synth_weights(Y, X, demean = TRUE)  # trends: the new default

best_single <- min(apply(X, 2, function(col) rmse(col, Y)))

fit_tr <- as.vector(X %*% w_tr)
off <- mean(Y) - mean(fit_tr)

tibble(
  game = tg,
  n_donors = ncol(X),
  n_pre_weeks = nrow(X),
  rmse_gd_levels = rmse(as.vector(X %*% w_gd), Y),
  rmse_qp_levels = rmse(as.vector(X %*% w_qp), Y),
  rmse_best_donor = best_single,
  rmse_qp_trend = rmse(fit_tr - mean(fit_tr), Y - mean(Y)), # what DiD actually needs
  rmse_qp_trend_levelrestored = rmse(fit_tr + off, Y),
  # Weight diagnostics
  gd_max_w = max(w_gd),
  gd_n_eff = sum(w_gd > 0.01),
  gd_is_uniformish = max(w_gd) - min(w_gd) < 0.02,
  qp_max_w = max(w_qp),
  qp_n_eff = sum(w_qp > 0.01)
)
})

c1 %>% summarise(
  games = n(),
  mean_rmse_gd_levels = round(mean(rmse_gd_levels), 2),
  mean_rmse_qp_levels = round(mean(rmse_qp_levels), 2),
  mean_rmse_best_donor = round(mean(rmse_best_donor), 2),
  mean_rmse_qp_trend = round(mean(rmse_qp_trend), 2),
  # How often is the old solver worse than the best single donor?
  pct_gd_worse_than_best_single = round(100 * mean(rmse_gd_levels > rmse_best_donor), 1),
  pct_gd_weights_uniform = round(100 * mean(gd_is_uniformish), 1)
)

## # A tibble: 1 x 7
##   games mean_rmse_gd_levels mean_rmse_qp_levels mean_rmse_best_donor
##   <int>         <dbl>         <dbl>         <dbl>
## 1      8          6.96          1.86          3.5
## # i 3 more variables: mean_rmse_qp_trend <dbl>,
## #   pct_gd_worse_than_best_single <dbl>, pct_gd_weights_uniform <dbl>
c1

## # A tibble: 8 x 13
##   game n_donors n_pre_weeks rmse_gd_levels rmse_qp_levels rmse_best_donor
##   <chr>   <int>   <int>         <dbl>         <dbl>         <dbl>
## 1 Yoku's Isl~ 113     25          5.37          2.83          3.86
## 2 Transistor  54     25          6.55          0.0892         0.827

```

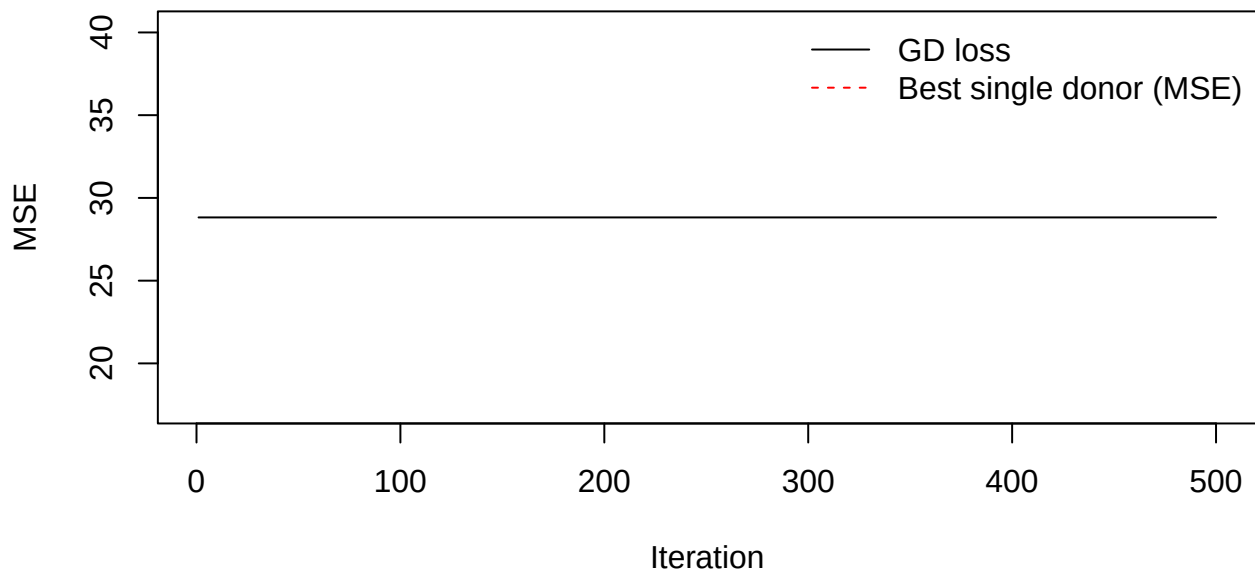
```
## 3 Unrailed!          97          25          10.4          3.27          4.00
## 4 The Elder ~       278          25          9.16          1.08          3.79
## 5 Dungeons 3        108          25          6.73          3.26          5.17
## 6 Call of Ct~       132          25          8.87          2.87          4.77
## 7 Golden Lig~       245          25          5.53          0.275         2.27
## 8 SUPERHOT          127          25          3.04          1.25          3.33
## # i 7 more variables: rmse_qp_trend <dbl>, rmse_qp_trend_levelrestored <dbl>,
## #   gd_max_w <dbl>, gd_n_eff <int>, gd_is_uniformish <lgl>, qp_max_w <dbl>,
## #   qp_n_eff <int>
```

Convergence Trace

The loss path for a single treated game, with the best single donor's MSE drawn as a reference line. A converged solver should end at or below that line.

```
d <- extract_donor_matrix(c1_games[1], 180, floor(180 / 7), "price")
if (!is.null(d)) {
  w_trace <- solve_synth_weights_gd(d$Y, d$X, trace = TRUE)
  plot(attr(w_trace, "loss"), type = "l",
       xlab = "Iteration", ylab = "MSE",
       main = paste0("Old GD loss path - ", d$game))
  abline(h = min(apply(d$X, 2, function(col) mean((col - d$Y)^2))),
        col = "red", lty = 2)
  legend("topright", legend = c("GD loss", "Best single donor (MSE)"),
        col = c("black", "red"), lty = c(1, 2), bty = "n")
}
```

Old GD loss path – Yoku's Island Express



Reading the 90-Day Price Result

The one significant price estimate appears at the 90-day window. These three views pull the relevant rows from tables already shown above, so the estimate, the fit quality behind it, and its pre-trend test can be read together.

```

comparison %>%
  filter(outcome == "price") %>%
  select(window, method, estimate, std.error, ci_lo, ci_hi, p.value, sig) %>%
  mutate(across(where(is.numeric), ~ round(.x, 3)))

## # A tibble: 6 x 8
##   window method          estimate std.error  ci_lo ci_hi p.value sig
##   <dbl> <chr>              <dbl>    <dbl>  <dbl> <dbl>  <dbl> <chr>
## 1    60 Stacked DiD          0.499    0.277 -0.044 1.04   0.072 ""
## 2    60 Synthetic Control DiD 0.348    0.248 -0.137 0.834  0.161 ""
## 3    90 Stacked DiD          0.475    0.231  0.023 0.927  0.04  "*"
## 4    90 Synthetic Control DiD 0.53     0.207  0.125 0.936  0.011 "*"
## 5   180 Stacked DiD          0.138    0.199 -0.253 0.529  0.49  ""
## 6   180 Synthetic Control DiD 0.233    0.196 -0.151 0.618  0.236 ""

# Fit quality behind each price estimate
fit_comparison %>% filter(matched_on == "price")

## # A tibble: 3 x 7
##   window matched_on stacked_trend_rmse n_matched synth_trend_rmse n_synth
##   <dbl> <chr>              <dbl>    <int>          <dbl>    <int>
## 1    60 price           2.46      255          0.737      255
## 2    90 price           3.01      244          1.09       244
## 3   180 price           3.79      227          2.01       227
## # i 1 more variable: synth_better_by <dbl>

# Pre-trend tests behind each price estimate
pretrends %>% filter(label == "Price (USD)")

## # A tibble: 6 x 7
##   label          window method          chi2    df p_value flag
##   <chr>          <dbl> <chr>          <dbl> <int>  <dbl> <chr>
## 1 Price (USD)    60 Stacked DiD          9.61     7  0.212  ok
## 2 Price (USD)    60 Synthetic Control DiD 7.27     7  0.402  ok
## 3 Price (USD)    90 Stacked DiD         13.2    11  0.278  ok
## 4 Price (USD)    90 Synthetic Control DiD 12.5    11  0.327  ok
## 5 Price (USD)   180 Stacked DiD         30.7    24  0.161  ok
## 6 Price (USD)   180 Synthetic Control DiD 43.7    24  0.0083 PRE-TREND

```